

NUS Orbital 2024
Milestone 3 Report



Team 6202

Chia Ming Han

Brian Chan Shi Yuan

Content

Posters	4
Preview of Web-App	8
Proof-of-Concept Videos	8
Link to GitHub Projects	8
Proposed Level of Achievement	8
Motivation	7
Aim	8
User Stories	8
Features	9
User Account Authentication [Completed]	9
CryptoCurrency Portfolio Table [Completed]	12
CryptoCurrency Portfolio PieChart [Completed]	17
Record of Sold Assets & Table [Completed]	19
Transaction History table [Completed]	21
ProfitLoss BarChart [Completed]	22
Top Finance News[Completed]	25
CryptoCurrency Live Prices Table [Completed]	26
Individual Coins Live Price Chart [Completed]	29
System Design	32
Application Design Flow	34
Model Entity Relationship Diagram (ERD)	34
Unified Modelling Language (UML) Diagrams	37
Tech Stack	40
Testing	41
User Testing	41
Developer Testing	42
Automated Testing	43
Software Engineering Practices	47
Modular Architecture	47
Testing Strategies	47
Continuous Improvement	48
Software Engineering Principles	49
Project Log	

Poster

CRYPTOSTASH

PROJECT AIM

Our project aims to help individuals consolidate their cryptocurrency holdings across various exchange platforms, providing users with a one stop application for all things crypto

PORTFOLIO DIVERSITY



Displays percentage of cryptocurrency out of total portfolio

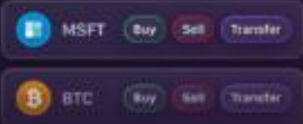
WHAT MAKES US UNIQUE?

We display cryptocurrency prices in both fiat and other forms of cryptocurrency.

This provides users with a multitude of ways to view the net worth of their portfolio.

ADD MANUAL TRANSACTIONS TO PORTFOLIO

Allows users to key in their coins bought/ sold to allow accurate vizualization under CONSOLIDATED PORTFOLIO



NEWS FEED

Keeps users informed on news affecting crypto prices



CONSOLIDATED PORTFOLIO

Allows users to visualize their coin prices from various platforms simultaneously.



MULTICURRENCY SUPPORT

Displays total assets in terms of BOTH Fiat and Cryptocurrency. Allows for quantitative comparison between Crypto & Commodities.



TECHSTACK



INTRODUCTION

CRYPTOSTASH

TEAM INFORMATION

Our team consist of Brian and Minghan, 2 Year 1 students who boasts an avid interest in Cryptocurrency.

PROJECT OBJECTIVES

Our project aims to help individuals consolidate their cryptocurrency holdings across various exchange platforms, providing users with a one stop application for all things crypto

SYSTEM DESIGN

SWE PRINCIPLES

DRY: Don't Repeat Yourself
By storing the code for common functions in files, we are able to call the file as and when the function is required. This saves us time and prevents us from repeating ourselves.

KISS: Keep It Simple, Stupid!
Instead of typing all the code in one large file, we try to split the code into multiple files, calling the functions only when required. This helps to ensure that the longest code is around 40 lines, keeping it simple.



NEWS FEED
Keeps users informed on news affecting crypto prices



CONSOLIDATED PORTFOLIO
Allows users to visualize their coin prices from various platforms simultaneously.

TECH STACK

We use
React.js
for our
frontend



Node.js
for our
backend



HTML CSS for the styling
and design of our
webpage



php for
linking to
database



EVALUATION

TESTING

To test out our application, utilised Unit & Integration, as well as Usability testing on our friends.



RESULTS

1. UI/UX Mismatch

As we develop the application according to function, these functions usually have different user interface as it is referenced to code done by different people

2. Long time taken to fetch data

Fetching data from multiple sources can slow down the application, especially if not optimized.

3. API requests exceeded

As we are using multiple APIs in our application, the limit was exceeded when the functions were combined, causing our users to not be able to view the data as desired



CONCLUSION

INTERPRETATION OF RESULTS

The mismatch of UI means that more work has to be done on the frontend to integrate change the interfaces such that they are more coherent.

Exceeding API requests means that a new API has to be used.

FUTURE PLANS?

Improving frontend code to make the UI more congruent

Using new APIs which allows for more calls to be made

Continue adding the remaining functions such as adding manual transactions, etc



CRYPTOSTASH

KEY FEATURES

INTERACTIVE Coin TABLE

Name	Price	24h Change	Market Cap
Bitcoin	\$42,150	+2.5%	\$845,000,000,000
Ethereum	\$2,800	+1.8%	\$336,000,000,000
Cardano	\$1.10	+0.5%	\$24,000,000,000
Bitcoin Cash	\$235	+0.2%	\$4,500,000,000
XRP	\$0.62	+0.1%	\$3,100,000,000
Litecoin	\$135	+0.3%	\$2,700,000,000

Search and filter cryptocurrencies.
View real-time prices, market cap, and 24h changes.
Clickable rows for detailed coin information.
Change Currency viewing



SYSTEM DESIGN

DYNAMIC CHARTS



CONSOLIDATED PORTFOLIO

Allows users to visualize their coin prices from various platforms simultaneously.

Name	Price	24h Change	Market Cap
Bitcoin	\$42,150	+2.5%	\$845,000,000,000
Ethereum	\$2,800	+1.8%	\$336,000,000,000
Cardano	\$1.10	+0.5%	\$24,000,000,000
Bitcoin Cash	\$235	+0.2%	\$4,500,000,000
XRP	\$0.62	+0.1%	\$3,100,000,000
Litecoin	\$135	+0.3%	\$2,700,000,000

NEWS FEED
Keeps users informed on news affecting crypto prices



Profit & Loss Tracker & Receipts

Transaction	Amount	Price	Fee	Net
Buy Bitcoin	1.5	\$28,100	\$42.15	\$42,142.85
Sell Bitcoin	0.5	\$28,100	\$14.05	\$14,042.85
Buy Ethereum	1.0	\$2,800	\$8.40	\$2,791.60
Sell Ethereum	0.5	\$2,800	\$4.20	\$1,395.80

Profit/Loss Form: Record and submit cryptocurrency transactions.

Transaction History: Track and review historical transactions.

USER EXPERIENCE ENHANCEMENT

- Confirmation Modal: Provides options for adding or replacing existing cryptocurrencies.
- Private Routes: Secure access to authenticated pages.

TECHSTACK



Preview of Web-App

https://github.com/minghan101/cryptostash_final

Proof-of-Concept Videos

https://drive.google.com/drive/u/0/folders/1temiALn9MeJzfk_cD1DB7Wb20UqV5f2y

Link to Github Project

https://github.com/minghan101/cryptostash_final

Proposed Level of Achievement

Apollo

Motivation

The rapid rise in cryptocurrency adoption has led to an explosion in the number of exchange platforms—nearly 600 at present. While this proliferation of platforms has made cryptocurrency more accessible to the general public, it has also introduced a significant challenge: managing and tracking diverse cryptocurrency holdings spread across multiple exchanges.

As users engage with different exchanges, they face the complexity of consolidating their portfolios, which may be represented in various fiat currencies, commodities like gold, or other cryptocurrencies. This fragmentation makes it difficult to get a clear and comprehensive view of one's overall cryptocurrency assets.

CryptoStash addresses these challenges by offering a streamlined solution for portfolio management. It empowers users to centralize their cryptocurrency holdings in one place, allowing for easier tracking and management. With CryptoStash, users can:

1. **Consolidate Holdings:** Integrate and view all their cryptocurrency assets across different exchanges in one unified dashboard.
2. **Multi-Currency Representation:** Track portfolio values in various fiat currencies, commodities (such as gold), or other cryptocurrencies, providing flexibility and clarity in asset evaluation.
3. **Interactive Dashboard:** Access an intuitive and interactive dashboard that offers features like real-time news updates and a comprehensive portfolio summary, enabling users to view and manage their holdings with greater ease.

By consolidating diverse assets and offering real-time insights, CryptoStash simplifies the management of cryptocurrency portfolios, making it easier for users to make informed decisions and stay updated on their investments.

Aim

Our aim is to revolutionize the way users manage and track their cryptocurrency assets by providing a platform that is simple, efficient, and user-friendly. **CryptoStash** strives to:

1. **Simplify Asset Consolidation:** Offer a seamless solution for users to aggregate their diverse cryptocurrency holdings from various exchanges into a single, coherent view. This eliminates the complexity of managing multiple accounts and provides a unified perspective of their investments.
2. **Enhance Portfolio Visibility:** Deliver a comprehensive overview of the entire portfolio, enabling users to quickly assess their total assets, track performance, and make informed decisions with minimal effort. The platform is designed to save users time and streamline the process of portfolio management.
3. **Provide Real-Time News and Insights:** Integrate a news feature to keep users informed about the latest developments in the cryptocurrency world. By presenting relevant news and updates, CryptoStash helps users stay abreast of market trends and make well-informed investment decisions based on current events.
4. **Offer Customization and Flexibility:** Allow users to tailor their experience with the option to manually add transactions and adjust settings according to their personal preferences. This flexibility ensures that the platform can adapt to individual needs and provide a more personalized management experience.

Through these aims, CryptoStash is committed to delivering a powerful tool that simplifies cryptocurrency portfolio management, enhances decision-making with timely information, and accommodates individual user needs for a more customized experience.

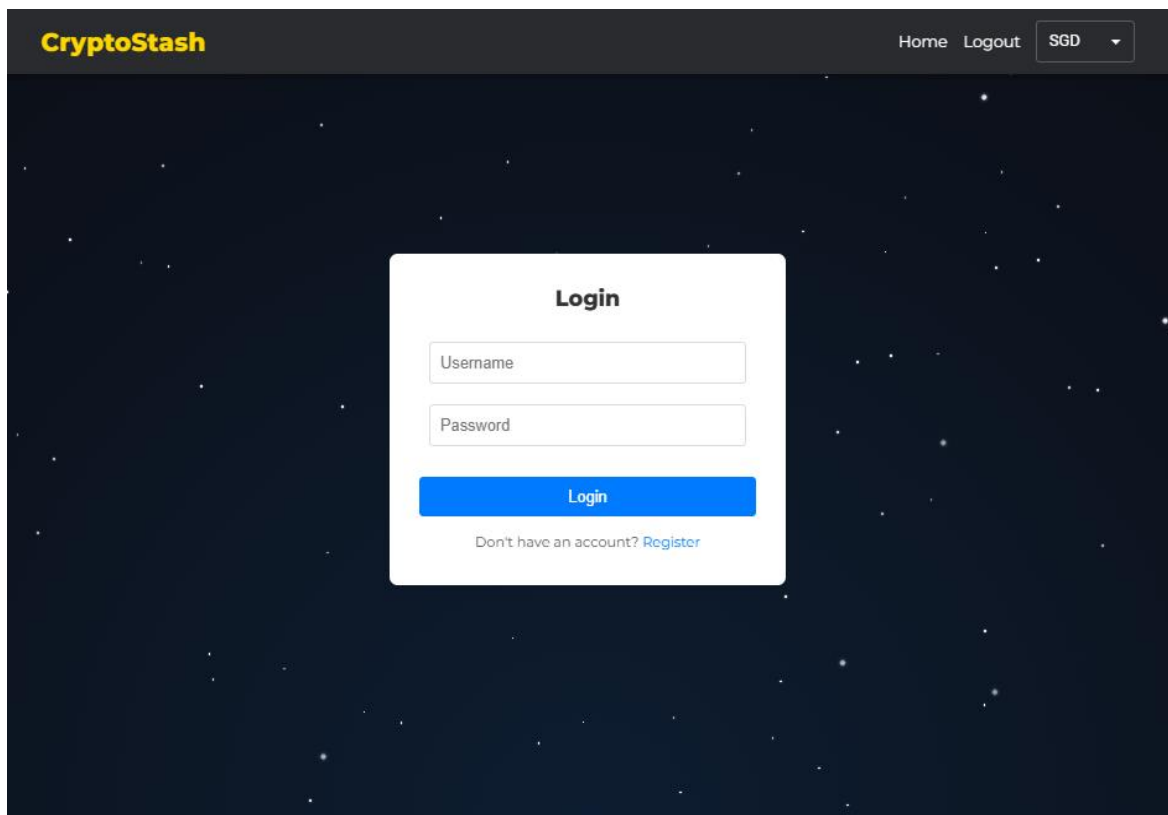
User Stories

1. **As a cryptocurrency investor,** I want to be able to register for a CryptoStash account using either my email address or social media accounts, so I can quickly and easily access my account.
2. **As a cryptocurrency investor,** I want to see a consolidated view of my cryptocurrency portfolio across multiple exchange platforms, so I can efficiently track and manage all my investments in one place.
3. **As a user,** I want to manually log my transactions into CryptoStash, so I can maintain an accurate and comprehensive record of my buying and selling activities.
4. **As a user,** I want to view interactive dashboards that display my portfolio's performance over time, so I can gain insights into how my investments are performing and make informed decisions.
5. **As a user,** I want to see the equivalent conversion price of my cryptocurrencies to fiat currencies, such as USD or EUR, and commodities like gold or silver, so I can understand the value of my investments from different perspectives.
6. **As a cryptocurrency investor,** I want to convert my cryptocurrency holdings into other cryptocurrencies or fiat currencies within the platform, so I can manage and rebalance my portfolio with ease.

7. **As a user**, I want to read the latest news and updates related to the cryptocurrency market within the CryptoStash platform, so I stay informed about market trends and developments that could impact my investments.
8. **As a user**, I want to customize my CryptoStash dashboard to display the information most relevant to me, so I can have a personalized and tailored experience that meets my individual needs.
9. **As a user**, I want to analyze my transaction history and view detailed reports on my profits and losses, so I can assess the performance of my investments and make strategic decisions based on historical data.
10. **As a cryptocurrency investor**, I want to receive notifications or alerts about significant changes in the market or my portfolio, so I can respond promptly to important developments.

Features

1. User Account Authentication



The screenshot displays the login interface of the CryptoStash platform. The header includes the 'CryptoStash' logo and navigation links for 'Home', 'Logout', and a currency selector 'SGD'. The main content area features a central white login box with the title 'Login'. Inside this box, there are input fields for 'Username' and 'Password', followed by a blue 'Login' button. A link for users without an account, 'Don't have an account? Register', is positioned at the bottom of the login box.

The screenshot shows the 'Register' form on the CryptoStash website. The form is centered on a dark blue background. It includes a title 'Register', a username input field with the text 'test2', a password input field with masked characters, a blue 'Register' button, and a link 'Already have an account? Login' below the button. The top of the page features the 'CryptoStash' logo and navigation links for 'Home', 'Logout', and a currency dropdown menu set to 'INR'.

Description: The User Account Authentication feature secures the access to CryptoStash by allowing users to create and manage their accounts through various authentication methods.

Details:

- a. **Registration.** Users can sign up for an account using their username and password in a very simple format. This flexibility caters to different user preferences and speeds up the sign-up process. Miscellaneous features such as allowing users to sign in with their email and other accounts can be explored, but is currently incomplete.
- b. **Login Process.** Once registered, users can log in using their credentials. For security, login is managed through JSON Web Tokens (JWT). Upon successful login, a JWT is generated, allowing users to authenticate their sessions without re-entering credentials.
- c. **Password Security.** Passwords are hashed using bcrypt before storage in the database. This hashing ensures that even if the database is compromised, user passwords remain secure. The hashing process includes a salt to further protect against brute-force attacks.
- d. **Password Reset (Incomplete).** Users who forget their passwords can request a reset. The system generates a secure reset link, which users can use to set a new password, ensuring their account remains accessible yet secure.

Purpose: This feature ensures that user data and accounts are protected from unauthorized access. By supporting multiple authentication methods and employing secure password handling practices, it enhances overall user security and accessibility.

Implementation Philosophy

Our authentication system integrates various methods to cater to different user preferences and enhance accessibility. Users can log in using traditional email and password combinations or through social media platforms like Google and Facebook. This flexibility ensures a broader range of options for users to choose from.

For secure session management, we employ JSON Web Tokens (JWT). JWTs are stateless, meaning that all necessary information is embedded within the token itself. This approach minimizes the need for server-side storage and enhances security.

Regarding password management, we utilize bcrypt to hash passwords before storing them in our database. This method adds an extra layer of security by ensuring that passwords are not stored in plaintext.

To facilitate password resets, we have implemented an email-based mechanism. When users need to reset their passwords, they receive a secure link via email. This process not only maintains account security but also offers a user-friendly experience.

Implementation Challenges: Securing JWTs against interception or tampering was a significant challenge. We addressed this by employing robust storage and transmission practices. Integrating with various social media platforms involved handling different OAuth implementations and API constraints, necessitating careful management of tokens and user data in accordance with each platform's policies. Ensuring that the bcrypt library provided robust security while balancing performance was crucial, as was managing data integrity and synchronization across different authentication methods.

Tech Stack:

1. Frontend:

- a. **React.js:** Utilized for building the user interface components for authentication, including login forms and registration pages. React's component-based architecture allows for a responsive and dynamic UI.
- b. **Redux:** Used for state management to handle authentication states, such as login, registration, and session management, across different components.

2. Backend:

- a. **Node.js:** Powers the server-side logic for handling authentication requests, session management, and integration with external authentication providers.
- b. **Express.js:** A web framework for Node.js that simplifies the creation of RESTful APIs for authentication, password reset, and user management.
- c. **JWT (JSON Web Tokens):** Employed for secure session management. Tokens are issued upon successful login and are used for user authentication in subsequent requests.

3. Database:

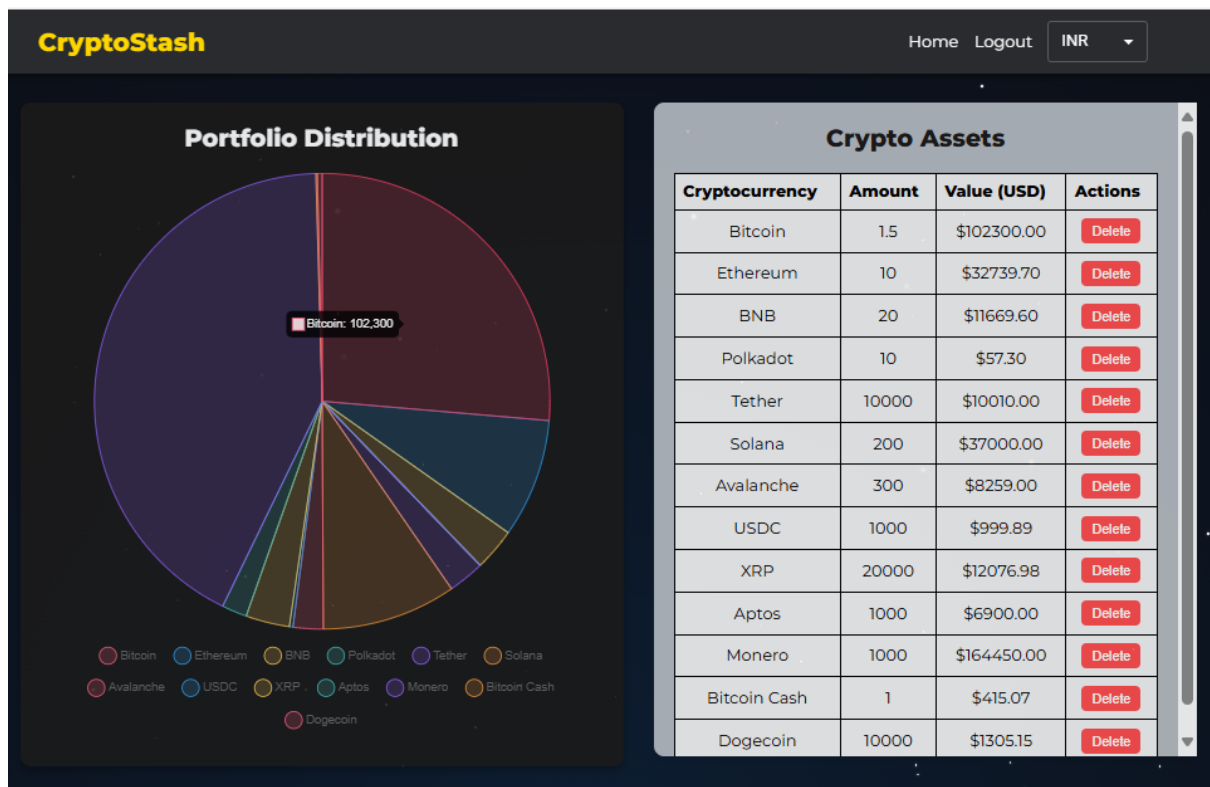
- a. **MongoDB:** Stores user credentials and authentication data. User profiles, including hashed passwords and session tokens, are stored in MongoDB collections.
- b. **bcrypt:** Used for hashing passwords before storing them in the database, adding a layer of security against unauthorized access.

Possible Areas for Improvement:

To enhance security, implementing Multi-Factor Authentication (MFA) could add an extra layer of protection beyond traditional password-based methods. Additionally, refining OAuth scopes to limit access to only necessary information can improve both privacy and security. For user experience, optimizing social media login processes can reduce friction and make it easier for users to sign in. Improving the onboarding flow with a step-by-step guide or progress tracker can help users complete their profiles more efficiently. Regarding error handling, providing more detailed and user-friendly error messages during authentication can assist users in troubleshooting issues more effectively. In terms of performance, optimizing token refresh mechanisms can reduce latency and improve login times.

2. CryptoCurrency Portfolio Table (Completed)

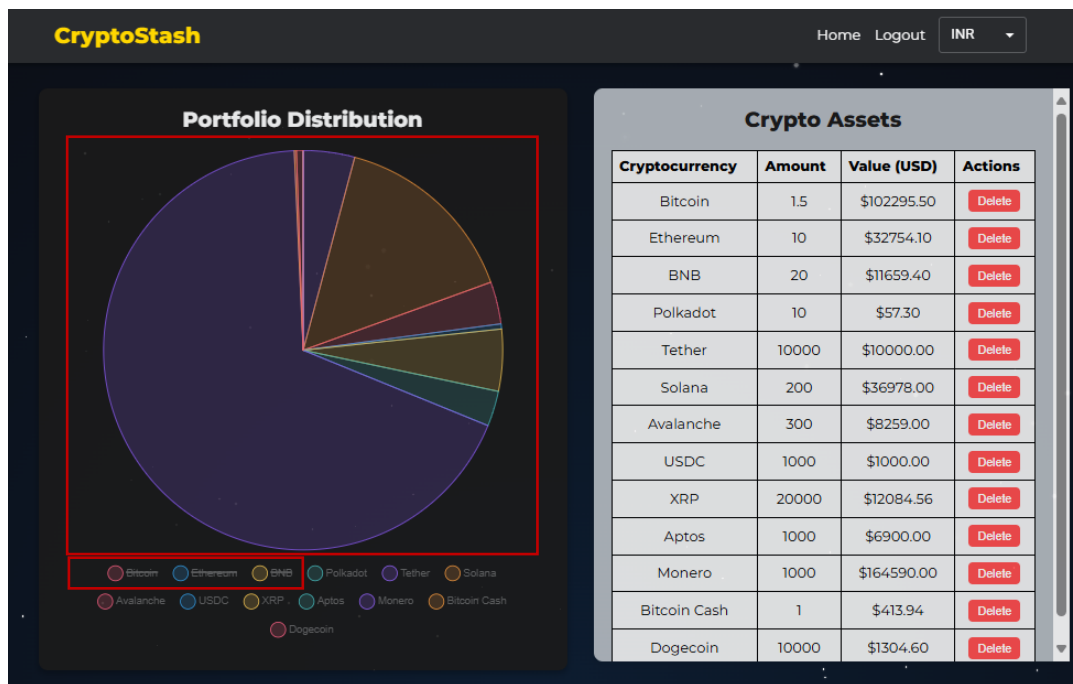




Description: The Cryptocurrency Portfolio Table provides users with a detailed tabular view of their cryptocurrency holdings, offering insights into their investments.

Details:

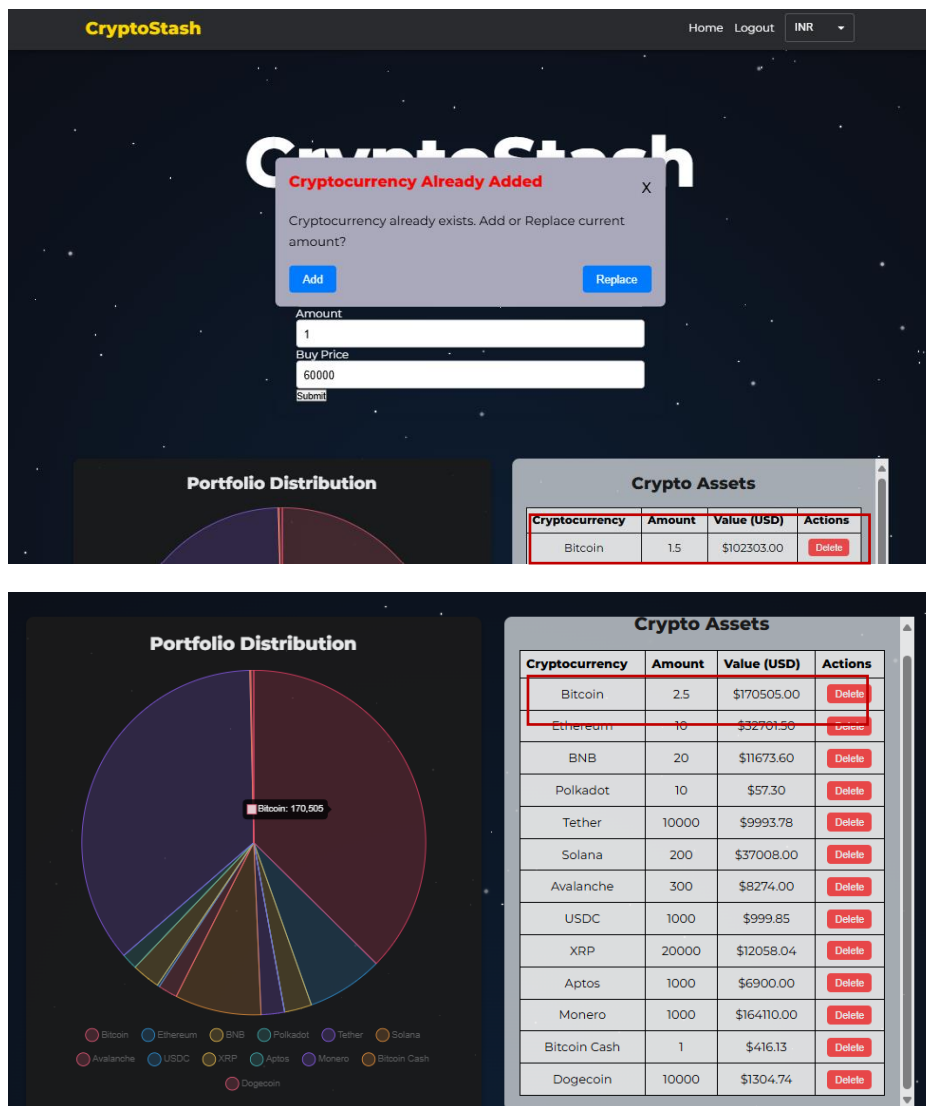
- Table Structure.** The table includes columns for cryptocurrency names, quantities held, buy prices, and current values. Each row represents a different cryptocurrency.
- Data Integration.** Data is fetched from the database and displayed in real-time, reflecting the most current holdings and values.
- Update and Management.** Users can add new assets, update existing ones, or delete them based on their trading activities. This management is reflected in the table immediately. Users can hide coins from the portfolio, which causes the chart to change automatically.



- d. **Sorting and Filtering (Incomplete).** Users can sort the table by asset name, quantity, or value and filter to view specific cryptocurrencies or holdings within a certain range.
- e. **Deletion.** Wrong entries are deleted from the table and the database.



- f. **Dropdown list.** Validation of input checked by dropdown list which fetches the available from Coin Gecko API. The server that uses this crypto name keyed in by user, fetches it's ID via the Coin Gecko API and then looks for the live prices from Coin Gecko Live Prices API.



- g. **Confirm Screen.** When user keys in duplicated cryptocurrency names, they will be prompted to either “Add” or “Replace” their current existing cryptocurrency in their database. Clicking “Add” will directly add to the crypto amount and hence the value, causing the chart to change accordingly.

Purpose: The portfolio table helps users keep track of their holdings in a structured and organized manner. It provides a clear snapshot of their investments, enabling effective portfolio management and decision-making.

Implementation Philosophy: The cryptocurrency portfolio table dynamically retrieves and displays up-to-date data from the database, ensuring users see the latest information. We implemented sorting and filtering features to enhance usability and allow users to customize their views according to their preferences. The table integrates with live data sources for real-time updates, reflecting market changes promptly.

Implementation Challenges: Data synchronization with live market data was a key challenge, requiring efficient mechanisms to handle data refreshes. Optimizing performance for large datasets involved employing efficient querying and indexing to prevent lag during

sorting and filtering operations. Balancing a clean, user-friendly interface with the need to display a comprehensive dataset required thoughtful design and continuous iteration.

Tech Stack:

1. Frontend:

- a. **React.js:** Used for rendering the portfolio table, enabling dynamic updates and interaction features like sorting and filtering.
- b. **Ant Design or Material-UI:** UI component libraries providing pre-built table components that offer a polished and responsive design.

2. Backend:

- a. **Node.js and Express.js:** Handle API requests for fetching and updating portfolio data. Routes are designed to handle CRUD operations for user portfolios.
- b. **WebSocket:** For real-time data updates, WebSocket connections are established to push live market data to the frontend.

3. Database:

- a. **MongoDB:** Stores portfolio data, including asset holdings and user configurations. Each user's portfolio is maintained in a collection with necessary indexing for efficient querying.

4. APIs:

- a. **Cryptocurrency Market APIs:** Integrated to fetch live price data and portfolio values. Examples include CoinGecko or CoinMarketCap APIs.

Challenges:

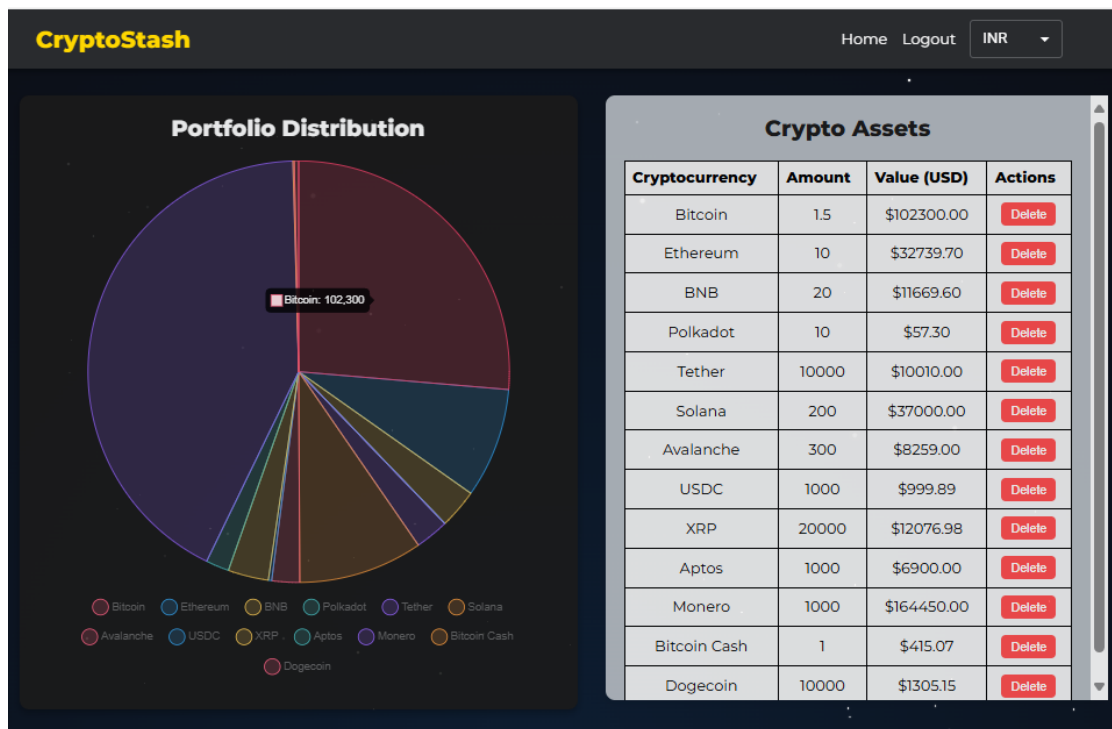
- 1. **Data Synchronization:** Ensuring real-time data updates and consistency with live market data.
- 2. **Performance Optimization:** Handling large datasets efficiently to ensure smooth user interactions and performance.

Possible Areas for Improvement:

In terms of user interface enhancements, allowing users to customize which columns are displayed or hidden can cater to individual preferences, while ensuring a responsive design will help the table adapt well to different screen sizes and devices. For performance optimization, implementing pagination or infinite scrolling can manage large portfolios more effectively and improve load times. Utilizing data caching strategies can reduce the number of API calls and speed up data retrieval. Advanced features such as more granular real-time

updates and improved search and filtering capabilities can further enhance user experience by allowing users to find specific assets quickly and control update frequencies.

3. CryptoCurrency Portfolio PieChart (Completed)



Description: The CryptoCurrency Portfolio PieChart visually represents the distribution of various cryptocurrencies within the user's portfolio.

Details:

- Chart Visualization.** The pie chart displays segments representing different cryptocurrencies, with each segment's size proportional to the asset's percentage of the total portfolio.
- Interactivity.** Users can hover over or click on chart segments to view detailed information about each cryptocurrency, such as its name, amount held, and value.
- Dynamic Updates.** The chart updates automatically as users' portfolio data changes, ensuring that the visual representation reflects the most current distribution.

Purpose: This visual tool enables users to quickly grasp the composition of their portfolio. It aids in understanding asset allocation and making informed decisions about adjusting their investments.

Implementation Philosophy: The pie chart visually represents the distribution of assets within a user's portfolio, facilitating quick understanding of portfolio composition. Interactive features, such as hovering over or clicking on chart segments, display detailed information,

enhancing the user experience. The chart updates in real-time to reflect changes in the portfolio, ensuring users always have access to current data.

Implementation Challenges: Maintaining data accuracy in the pie chart involved meticulous handling and calculation of portfolio distribution. Optimizing performance for real-time updates, especially with large portfolios, required employing efficient rendering techniques and data processing. Designing an intuitive and responsive interface for user interactions, such as hover and click events, was essential for providing meaningful insights.

Tech Stack:

1. Frontend:

- **React.js:** Used for rendering the pie chart component. React hooks manage state and updates for the chart.
- **Chart.js or D3.js:** Libraries used for rendering the pie chart. Chart.js provides simple and easy-to-use charting options, while D3.js offers advanced customization.

2. Backend:

- **Node.js and Express.js:** Handle data aggregation and provide the processed data to the frontend for rendering the chart.

3. Database:

- **MongoDB:** Stores the data required to generate the pie chart, such as asset distribution and historical data.

Challenges:

- **Data Accuracy:** Ensuring that the pie chart accurately reflects user portfolio data and updates in real-time.
- **Rendering Performance:** Optimizing chart rendering performance, especially with large datasets or complex visualizations.

Possible Areas for Improvement:

To enhance visualization, adding interactive features like tooltips or drill-down capabilities can provide users with more detailed information on each segment. Offering customization options, such as changing colors, chart types, or labels, can allow users to tailor the chart to their preferences. Performance-wise, optimizing chart rendering to handle large datasets without degrading performance is crucial. Additionally, enabling users to save and load different chart configurations based on their preferences can improve usability and provide a more personalized experience.

4. Record of Sold Assets & Table (Completed)

Crypto Asset	Amount	Buy Price	Sell Price	Date
Bitcoin		0	30000	
Ethereum		0	4000	
Litecoin	1	0	110	2024-27T00
Polkadot	1	0	10	2024-27T00
Polkadot	1	10	0	2024-27T11
Bitcoin	0.5	0	40000	2024-27T00
Polkadot	20	100	0	2024-28T00

Description: The Record of Sold Assets feature maintains a comprehensive log of all cryptocurrency sales, detailing each transaction's specifics.

Details:

- Sales Recording.** The system records essential details of each sale, including asset name, amount sold, sell price, and the date of the transaction.
- Historical Data.** Users can view historical sales data in a table format, allowing them to track past sales and assess their trading history.
- Transaction Analysis.** The table may include columns for profit/loss calculations based on the difference between buy and sell prices.
- Deletion.** Wrong entries can be deleted from the table and the database.

Purpose: This feature provides transparency and a historical record of sales transactions. It allows users to analyze past trades and evaluate their trading performance, facilitating better financial planning.

Implementation Philosophy: We ensure comprehensive logging of all sales transactions, including asset names, quantities, and prices, creating a complete history of sales activities. The table organizes sold assets, allowing users to review their trading history and assess performance over time. Integrated profit and loss calculations provide insights into trading performance.

Implementation Challenges: Efficiently storing and retrieving historical sales data while maintaining integrity required careful indexing and database design. Designing the table to be

informative and user-friendly involved balancing detailed information with ease of navigation. Accurate profit and loss calculations based on historical data demanded handling various edge cases and ensuring precision.

Tech Stack:

1. Frontend:

- **React.js:** Manages the display of the sold assets table and allows for user interactions such as sorting and filtering.
- **Ant Design or Material-UI:** Provides table components with features for managing and displaying historical data.

2. Backend:

- **Node.js and Express.js:** Manage API routes for fetching and recording sold asset data.
- **Mongoose:** An ODM (Object Data Modeling) library for MongoDB that helps manage data schemas and interactions for storing sold asset records.

3. Database:

- **MongoDB:** Stores records of sold assets, including transaction details and timestamps.

Challenges:

1. **Historical Data Management:** Efficiently managing and retrieving historical data, especially as transaction volumes grow.
2. **Data Integrity:** Ensuring that sold asset records are accurately maintained and displayed.

Possible Areas for Improvement:

For data management, incorporating functionality for bulk importing or exporting sold asset records can simplify data management. Implementing data integrity checks can ensure that records are accurate and complete. In terms of user interface, enhancing sorting and filtering options can make it easier to analyze sold asset data, while adding visual indicators can highlight significant transactions. Introducing trend analysis and insights into sold assets can help users understand patterns and make better investment decisions.

5. Transaction History Table (Completed)

Transaction Records				
Crypto Asset	Amount	Buy Price	Sell Price	Date
Bitcoin		0	30000	
Ethereum		0	4000	
Litecoin	1	0	110	2024-07-27T00:00:00.000Z
Polkadot	1	0	10	2024-07-27T00:00:00.000Z
Polkadot	1	10	0	2024-07-27T11:03:00.302Z
Bitcoin	0.5	0	40000	2024-07-27T00:00:00.000Z
Polkadot	20	100	0	2024-07-28T08:41:45.332Z
	4000			2024-07-27T00:00:00.000Z
	110			2024-07-27T00:00:00.000Z
	10			2024-07-27T00:00:00.000Z
	0			2024-07-27T11:03:00.302Z
	40000			2024-07-27T00:00:00.000Z
	0			2024-07-28T08:41:45.332Z
	0			2024-07-28T08:57:57.847Z
	000			2024-07-28T10:34:43.747Z

Description: The Transaction History Table presents a detailed log of all transactions conducted by the user, including both buy and sell actions.

Details:

- a. **Table Structure.** The table includes columns for transaction type (buy/sell), asset name, amount, price, and date. This structure helps users track all their trading activities comprehensively.
- b. **Sorting and Filtering (Incomplete).** Users can sort transactions by date, type, or asset, and filter to view specific types of transactions or date ranges.
- c. **Data Integrity.** Transactions are recorded with timestamps to ensure accurate chronological order and integrity of historical data.
- d. **Deletion.** Users can delete wrong entries from the database and the table, which will be reflected in the charts automatically.

Purpose: This feature enables users to maintain an organized record of all their trading activities, providing a clear view of their historical transactions and helping with performance analysis.

Implementation Philosophy: The transaction history table captures all user transactions, including both buys and sells, providing a complete record of trading activities. Users can sort and filter transactions based on various criteria, enhancing usability and enabling detailed analysis. Transactions are displayed in chronological order with timestamps to facilitate accurate tracking of trading history.

Implementation Challenges: Handling large volumes of transaction data while maintaining performance required efficient data management and retrieval strategies. Designing a table that is detailed yet easy to navigate necessitated balancing extensive data presentation with user experience considerations. Ensuring accurate recording and display of transaction details involved rigorous testing and validation.

Tech Stack:

1. Frontend:

- React.js:** Builds the user interface for displaying transaction history. React's component-based structure allows for dynamic updates.
- Ant Design or Material-UI:** Provides table components with features for sorting and filtering transaction data.

2. Backend:

- Node.js and Express.js:** Handle API requests for transaction history, including fetching, filtering, and sorting data.
- Mongoose:** Used to interact with MongoDB and manage transaction history records.

Database:

- MongoDB:** Stores transaction history with details such as transaction dates, amounts, and asset types.

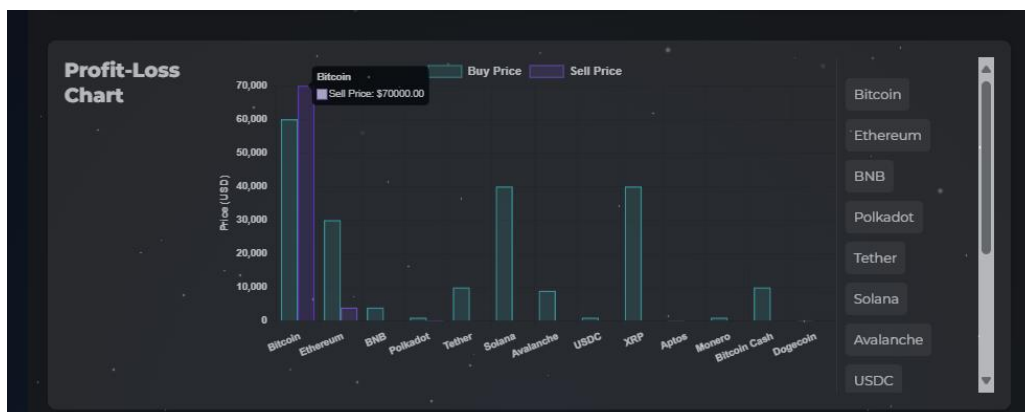
Challenges:

- Data Retrieval and Performance:** Efficiently retrieving and displaying large volumes of transaction data while maintaining performance.
- User Interface Design:** Designing a table that presents transaction data clearly and allows for easy navigation and analysis.

Possible Areas for Improvement:

User interface enhancements could include advanced filtering options, allowing users to search by various criteria such as date ranges, asset types, or transaction amounts. Providing export options to formats like CSV or Excel can facilitate offline analysis. To improve performance, implementing pagination or infinite scrolling can handle large volumes of transaction data without affecting performance. Integrating data visualization through charts or graphs can help users analyze transaction history trends more effectively.

6. Profit/Loss BarChart (Completed)





Description: The Profit/Loss BarChart visualizes the gains and losses from cryptocurrency transactions over a specified period.

Details:

- Chart Visualization.** The bar chart displays bars representing profit or loss for each asset or transaction period. Positive values indicate gains, while negative values represent losses.
- Time Period Analysis.** Users can view profits and losses over different time frames, such as weekly, monthly, or yearly, to analyze performance trends.
- Interactive Features.** Users can hover over or click on bars to get detailed information about specific transactions or periods. Users can click on the coin displayed, and it will hide the graph automatically to allow better visualization of other charts.

Purpose: This visualization helps users quickly assess their financial performance and understand how their investments have performed over time. It supports better decision-making by highlighting profitable and unprofitable trades.

Implementation Philosophy: The bar chart offers a visual summary of profits and losses over time, helping users assess their financial performance. Users can view profit and loss data across different time periods for trend analysis and performance evaluation. Interactive features such as hover effects and detailed tooltips enhance the user experience by providing additional context.

Implementation Challenges: Accurately representing profit and loss data while maintaining clarity and readability was a challenge, addressed through effective data aggregation and visualization techniques. Optimizing performance for rendering and updating the bar chart with large datasets required efficient data processing. Ensuring responsive interactions and meaningful insights with minimal lag involved careful design and testing.

Tech Stack:

- Frontend:**

- a. **React.js:** Manages the bar chart component, integrating with charting libraries to render data visualizations.
- b. **Chart.js or D3.js:** Used for creating bar charts, with Chart.js offering ease of use and D3.js providing advanced customization options.

2. Backend:

- a. **Node.js and Express.js:** Handle data processing and aggregation for profit/loss calculations.

3. Database:

- a. **MongoDB:** Stores historical data required for profit and loss calculations and charting.

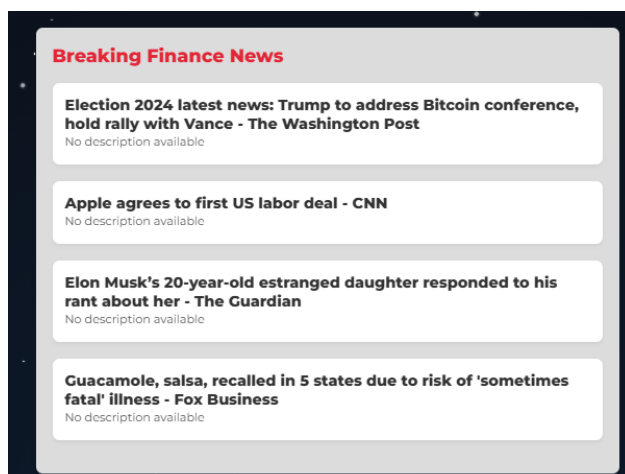
Challenges:

1. **Data Accuracy and Visualization:** Ensuring that the bar chart accurately reflects profit and loss data while maintaining clarity and usability.
2. **Performance:** Handling real-time updates and large datasets efficiently.

Possible Areas for Improvement:

To enhance visualization, adding interactive elements like tooltips or drill-down options can provide more details about individual bars. Allowing users to customize the chart's appearance, such as changing colors or time periods, can make the chart more user-friendly. For performance, ensuring that the bar chart performs well with large datasets and updates in real-time without lag is essential. Advanced analytics features, such as comparing profit and loss across different time periods or assets, can provide deeper insights into financial performance.

7. Top Finance News (Completed)



Description: The Top Finance News feature aggregates and displays the latest news related to the cryptocurrency market, keeping users informed about significant developments.

Details:

- a. **News Feed:** The feed shows headlines, summaries, and links to full articles from various financial news sources. It is updated regularly to include the most recent and relevant news.
- b. **Customization (Incomplete):** Users may have options to filter news based on their interests or specific cryptocurrencies.
- c. **Integration:** News data is fetched from reputable news providers to ensure accuracy and reliability.

Purpose: This feature keeps users updated on market trends, regulatory changes, and other news that could impact their investments. Staying informed helps users make better investment decisions.

Implementation Philosophy: The top finance news feature aggregates and displays the latest financial news relevant to the cryptocurrency market, keeping users informed about significant events. Users can filter news based on their interests to ensure a personalized and relevant news feed. Data is fetched from reputable sources to ensure accuracy and reliability.

Implementation Challenges: Integrating with various news APIs and managing real-time data fetching posed challenges related to API rate limits and data consistency. Ensuring the news feed remains up-to-date and relevant required continuous monitoring and integration with reliable news sources. Designing a clean, engaging news feed while ensuring accessibility and readability required careful UI/UX design.

Tech Stack:

1. **Frontend:**

- a. **React.js:** Manages the news feed display, allowing for dynamic updates and interaction.
- b. **Axios or Fetch API:** Used for making HTTP requests to fetch news articles from external sources.

2. **Backend:**

- a. **Node.js and Express.js:** Manage API requests for fetching and aggregating news data.

3. **APIs:**

- a. **News APIs:** Integrated with financial news APIs like NewsAPI or Alpha Vantage to fetch the latest news articles related to cryptocurrency.

Challenges:

1. **Data Fetching and Integration:** Handling API rate limits and ensuring the timely retrieval of relevant news data.
2. **Content Relevance:** Filtering and presenting the most relevant news to users based on their interests.

Possible Areas for Improvement:

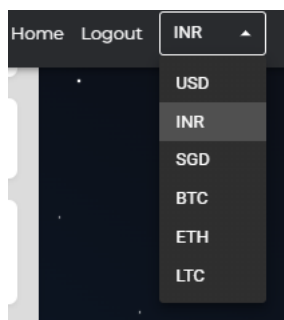
To improve content relevance, implementing algorithms to personalize news content based on user preferences and interests can enhance user engagement. Providing options to filter news by categories like market trends, regulatory changes, or technological developments can help users find the most relevant information. Performance can be improved by using caching mechanisms to reduce load times for fetching news articles. Additionally, integrating news with other features such as portfolio performance or transaction history can provide contextually relevant information.

8. CryptoCurrency Live Prices Table (Completed)

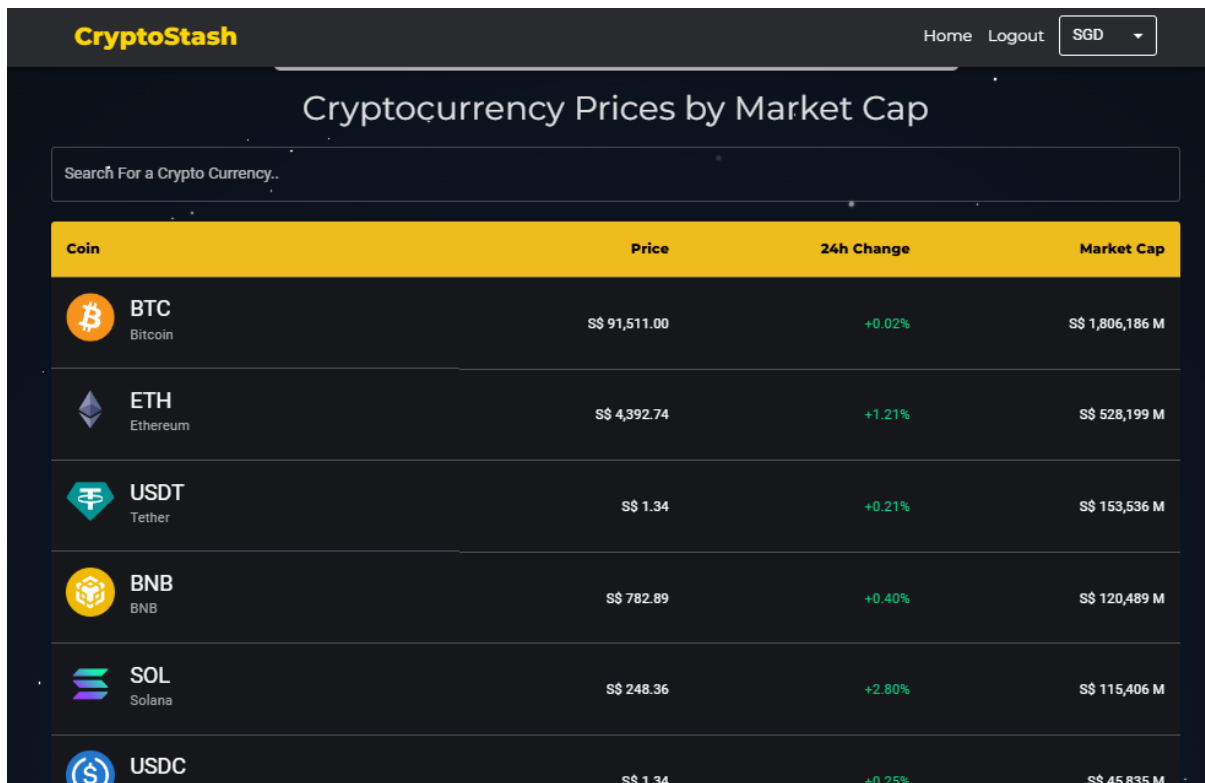
Description: The Live Prices Table provides real-time updates on the prices of various cryptocurrencies, allowing users to monitor market fluctuations.

Details:

- a. **Live Data:** The table displays current market prices of cryptocurrencies, fetched from live data sources or APIs.
- b. **Update Frequency:** Prices are refreshed periodically to ensure users receive the most up-to-date information.
- c. **Information of coins.** Upon clicking on each coins, it will display in detail the information of each coin, reflected in Coins Page.
- d. **Data Presentation:** Includes columns for cryptocurrency names, current prices, and price changes (if applicable).

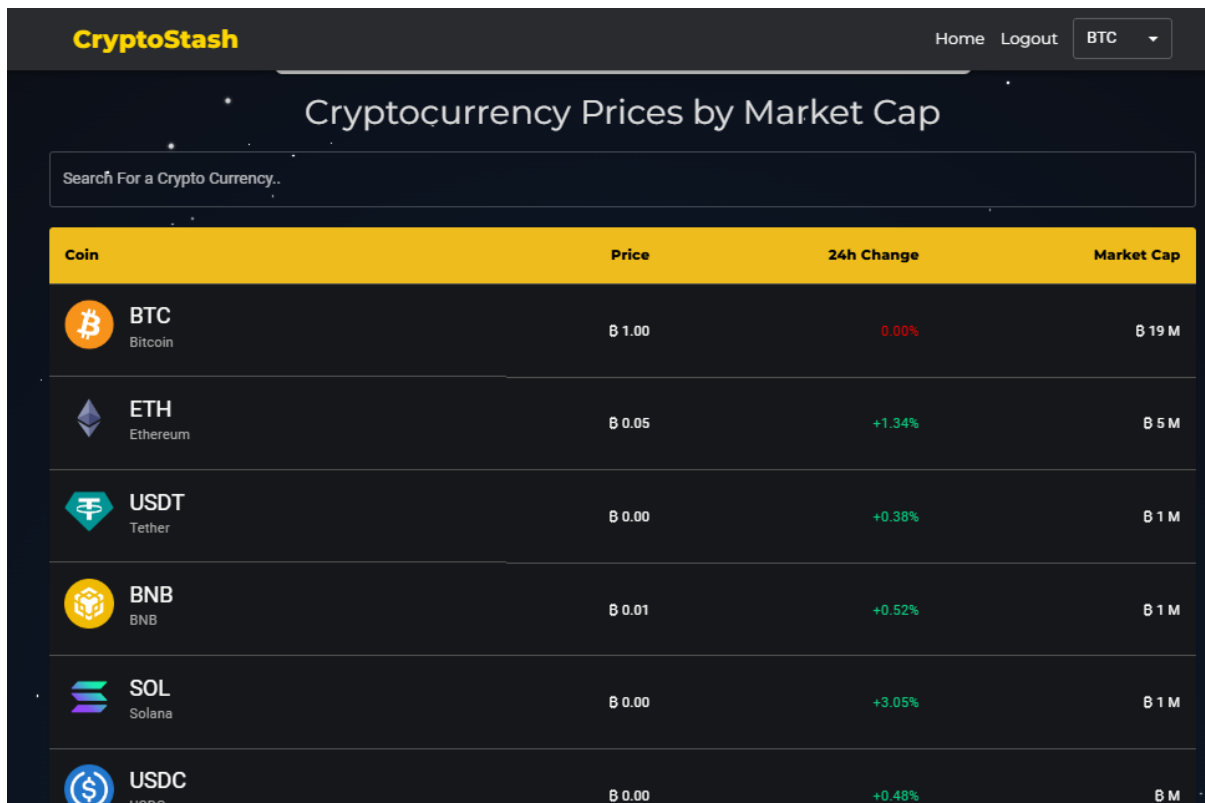


- e. **Currency Change.** As an enhancement, the currency can be reflected in many other forms: BTC/ ETH/ USD/ SGD/ INR etc. This will be reflected for the entire Coins Table.



The screenshot shows the CryptoStash website interface. At the top, there is a navigation bar with the CryptoStash logo, links for Home and Logout, and a dropdown menu currently set to SGD. The main heading is "Cryptocurrency Prices by Market Cap". Below this is a search bar labeled "Search For a Crypto Currency..". The main content is a table with four columns: Coin, Price, 24h Change, and Market Cap. The table lists six cryptocurrencies: BTC, ETH, USDT, BNB, SOL, and USDC, each with its respective icon, name, and full name. The prices are listed in SGD, and the 24-hour changes are shown in green, indicating an increase for all listed coins.

Coin	Price	24h Change	Market Cap
 BTC Bitcoin	S\$ 91,511.00	+0.02%	S\$ 1,806,186 M
 ETH Ethereum	S\$ 4,392.74	+1.21%	S\$ 528,199 M
 USDT Tether	S\$ 1.34	+0.21%	S\$ 153,536 M
 BNB BNB	S\$ 782.89	+0.40%	S\$ 120,489 M
 SOL Solana	S\$ 248.36	+2.80%	S\$ 115,406 M
 USDC USDC	S\$ 1.34	+0.25%	S\$ 45,835 M



The screenshot shows the same CryptoStash website interface, but the dropdown menu is now set to BTC. The table displays the same six cryptocurrencies, but the prices are now listed in BTC. The 24-hour changes are shown in red for BTC (0.00%) and green for the others, indicating an increase for ETH, USDT, BNB, SOL, and USDC.

Coin	Price	24h Change	Market Cap
 BTC Bitcoin	฿ 1.00	0.00%	฿ 19 M
 ETH Ethereum	฿ 0.05	+1.34%	฿ 5 M
 USDT Tether	฿ 0.00	+0.38%	฿ 1 M
 BNB BNB	฿ 0.01	+0.52%	฿ 1 M
 SOL Solana	฿ 0.00	+3.05%	฿ 1 M
 USDC USDC	฿ 0.00	+0.48%	฿ M

Purpose: This feature allows users to track real-time market prices and make timely trading decisions based on the latest data.

Implementation Philosophy: The live prices table provides real-time updates on cryptocurrency prices, allowing users to monitor market fluctuations and make informed decisions. Prices are fetched from live market data sources or APIs to ensure the information is current. Features for sorting and filtering enhance usability and enable users to focus on specific cryptocurrencies or data points.

Implementation Challenges: Managing the frequency of price updates to balance real-time accuracy with system performance was challenging, addressed through efficient data fetching and update mechanisms. Ensuring the accuracy of live price data and reflecting current market conditions required integrating with reliable data sources and handling potential discrepancies. Designing a table that effectively displays real-time data while maintaining performance was crucial.

Tech Stack:

1. Frontend:

- a. **React.js:** Builds the live prices table with real-time updates.
- b. **Ant Design or Material-UI:** Provides table components with features for displaying and interacting with live price data.

2. Backend:

- a. **Node.js and Express.js:** Handle API requests for fetching live price data from cryptocurrency market APIs.
- b. **WebSocket:** Used to push live price updates to the frontend in real-time.

3. Database:

- a. **MongoDB:** Stores user configurations and preferences related to live price data.

Challenges:

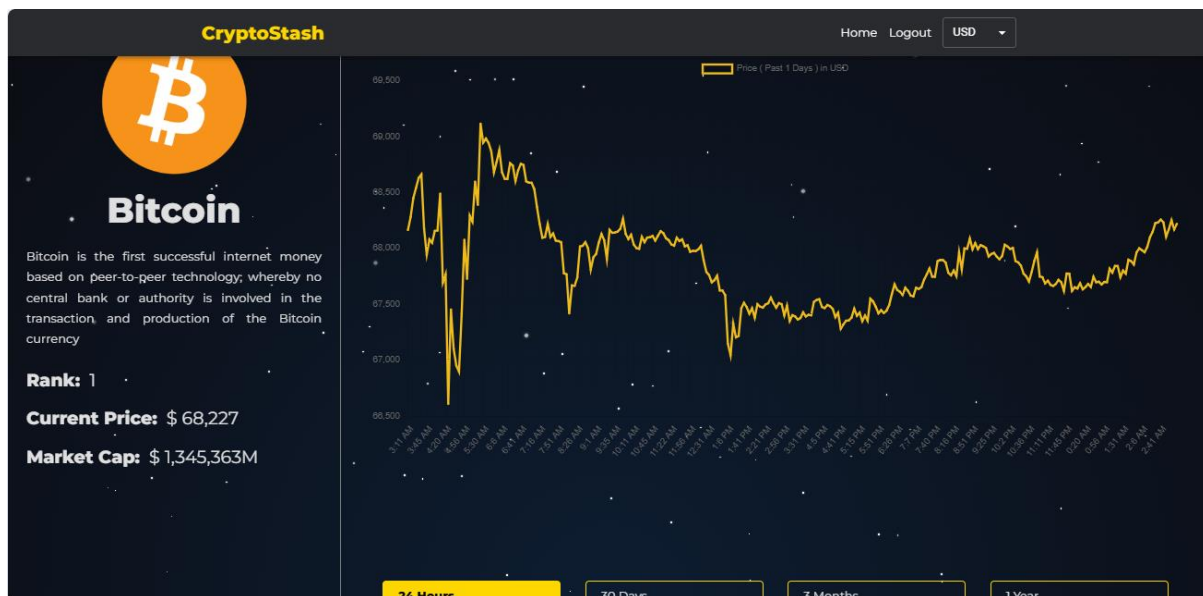
- 1. **Real-Time Data Handling:** Managing real-time data updates and ensuring the table displays current information without performance issues.
- 2. **Data Accuracy:** Ensuring that live price data is accurate and sourced from reliable market data providers.

Possible Areas for Improvement:

To handle real-time data effectively, implementing rate-limiting mechanisms can manage API rate limits and prevent service disruptions. Ensuring the accuracy of live price data and implementing fallback mechanisms in case of data source failures are crucial for reliable updates. User interface customization options, such as choosing which cryptocurrencies to display or adjusting the refresh rate, can enhance user experience. For performance,

optimizing the process for updating live prices can ensure smooth and responsive interactions.

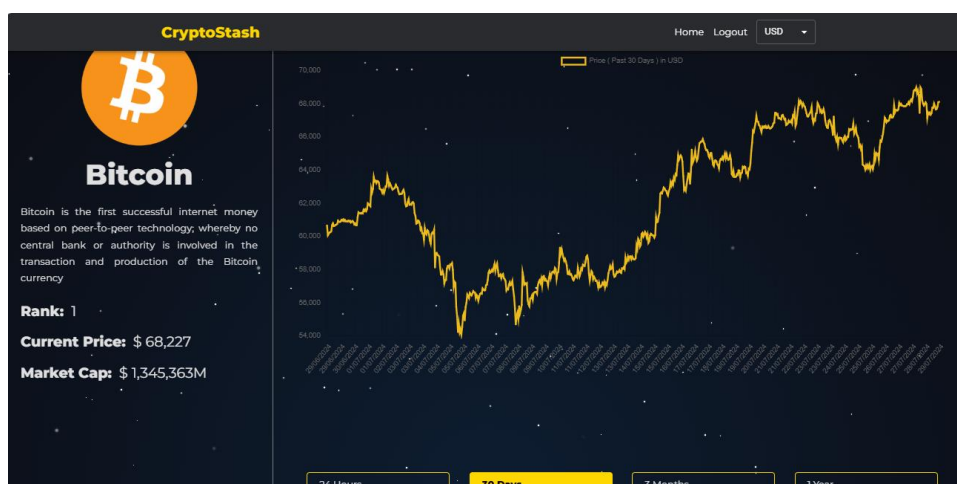
9. Individual Coins Live Price Chart (Completed)



Description: Can be accessed via clicking on the coins on Coins Table. The Individual Coins Live Price Chart provides a graphical representation of the price trend for selected cryptocurrencies over time. The cryptocurrency shown in the CoinTable can be clicked on and further details can be seen in the new coinspage.

Details:

- Chart Visualization:** The chart displays price movements for a chosen cryptocurrency, with options for different time frames (e.g., daily, weekly, monthly).
- Historical Data:** Users can view historical price data and analyze trends to inform trading strategies.



- c. **Interactivity:** Features like zooming and panning allow users to explore different parts of the chart for detailed analysis. Users can adjust the time range for them to see the live chart. (Can view in terms of Year/ Month/ Day/ Hour).

Purpose: This chart helps users analyze price trends for specific cryptocurrencies, enabling them to make informed decisions based on historical performance and current trends.

Implementation Philosophy: The live price chart offers a graphical representation of price trends for selected cryptocurrencies, enabling users to analyze market movements over time. Interactive features like zooming and panning allow for detailed exploration of historical data and current trends. The chart updates dynamically to reflect the latest price changes, ensuring users have access to current information.

Implementation Challenges: Accurately representing price trends and maintaining visual clarity required effective data visualization techniques and handling large datasets. Optimizing performance for real-time updates and historical data needed careful design to ensure smooth operation. Designing interactive features that enhance the user experience while keeping the chart functional and informative involved careful design and testing.

Tech Stack:

1. Frontend:

- a. **React.js:** Manages the individual coins' live price chart, providing a responsive and interactive user experience.
- b. **Chart.js or D3.js:** Used for rendering detailed price charts with historical data and real-time updates.

2. Backend:

- a. **Node.js and Express.js:** Process and aggregate price data for the charts.
- b. **WebSocket:** For real-time updates on price changes.

3. Database:

- a. **MongoDB:** Stores historical price data and user preferences for individual coins.

Challenges:

- 1. **Data Visualization:** Handling and visualizing large datasets and ensuring the chart remains responsive and informative.
- 2. **Performance:** Optimizing real-time updates and data rendering to prevent lag and ensure a smooth user experience.

Possible Areas for Improvement:

To enhance visualization, offering different types of charts, such as line charts or candlestick charts, can provide more in-depth analysis of price movements. Integrating historical price data with live data can give users a comprehensive view of price trends. Ensuring efficient handling of real-time data updates and maintaining chart performance with large datasets are critical for performance. Allowing users to select custom time frames or intervals for analyzing price data can further improve the chart's utility and user experience.

Summary

CryptoStash offers a comprehensive suite of features designed to enhance the user experience in managing cryptocurrency investments. Each feature is carefully crafted to provide users with essential tools for portfolio management, transaction tracking, financial analysis, and staying informed about market trends. By integrating robust data visualization tools, real-time market information, and user-friendly interfaces, CryptoStash aims to streamline portfolio management and support users in making well-informed investment decisions.

System Design of CryptoStash

The CryptoStash system integrates various software engineering design patterns and architectural principles to achieve modularity, scalability, and maintainability. Below is a refined explanation of how each design pattern and architectural choice is applied to the project, aligned with the specific features and tech stack used.

Model-View-Controller (MVC) Architecture

The MVC architecture is foundational to CryptoStash, dividing the application into three core components. The **Model** handles data and business logic, managing operations such as fetching, updating, and storing information. In CryptoStash, this includes models for user accounts, cryptocurrency portfolios, transactions, and live prices. The **View** is responsible for the presentation layer and user interface, encompassing elements like the Portfolio Table, Pie Chart, and Live Price Charts. For instance, the `CryptoCurrencyPortfolioTable` and `CryptoCurrencyPortfolioPieChart` components display data retrieved from the backend. The **Controller** acts as an intermediary, processing user actions such as logging in or adding transactions, and updates the Model with new data, which in turn refreshes the View. This separation of concerns facilitates independent management of user interface and business logic, supports scalability, and makes updates and maintenance more manageable.

Observer Pattern

The Observer pattern establishes a one-to-many dependency between objects, ensuring that when one object changes, all dependent objects are automatically updated. In CryptoStash, this pattern is used to handle real-time updates for live prices and portfolio changes. For example, when cryptocurrency prices change, components like the `CryptoCurrencyLivePricesTable` and `IndividualCoinsLivePriceChart` are updated to reflect the latest data. Similarly, when users modify their transactions, components such as the `TransactionHistoryTable` and `PortfolioSummary` are notified and refreshed. This pattern ensures dynamic updates across the application and reduces dependencies between data sources and the components that display the data.

Publisher-Subscriber Pattern

The Publisher-Subscriber (Pub-Sub) pattern allows for decoupled communication between components, where publishers send messages or events, and subscribers react to those events. In CryptoStash, this pattern is used for handling live updates for prices and news. For instance, the `TopFinanceNews` and `CryptoCurrencyLivePricesTable` components subscribe to live data feeds. When the backend (publisher) sends updates, these frontend components (subscribers) refresh their displays accordingly. The Pub-Sub pattern also applies to transaction updates, where components like the `RecordOfSoldAssetsTable` and `TransactionHistoryTable` are notified of changes and update their data. This pattern supports loose coupling between components and facilitates the addition of new data sources or modifications with minimal impact on existing code.

Singleton Pattern

The Singleton pattern ensures that a class has only one instance and provides a global point of access to that instance. In CryptoStash, this pattern is used for managing API clients that fetch live prices and news. For example, the CryptoPriceAPIClient is implemented as a singleton to handle all API requests for live price data, thereby optimizing resource usage and avoiding redundant connections. This pattern ensures consistency and efficient resource management across the application.

Factory Pattern

The Factory pattern offers an interface for creating objects while allowing subclasses to modify the types of objects created. In CryptoStash, the Factory pattern is utilized to instantiate various chart components based on user preferences. Depending on the type of portfolio analysis required, a factory may create a PieChart, BarChart, or LineChart. This approach allows for flexible and dynamic creation of different chart types, supporting easy extensibility with minimal code changes.

Strategy Pattern

The Strategy pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. In CryptoStash, this pattern is applied to currency conversion algorithms. Users can choose between different methods, such as direct conversion or average price. Each conversion strategy handles the conversion process differently, providing flexibility in how cryptocurrency values are displayed. The Strategy pattern encapsulates conversion logic, facilitating easier updates or additions of new methods while keeping the main application logic separate.

Implementation Challenges

Real-Time Data Handling: One major challenge is ensuring accurate and timely updates for live prices and news feeds. To address this, efficient data handling techniques are employed, such as using WebSockets for real-time updates and implementing caching strategies to minimize API calls.

Data Consistency: Maintaining consistency between user profiles, portfolios, and transaction histories is crucial. Solutions include implementing transactions and atomic operations for data updates, along with robust data validation and error handling mechanisms.

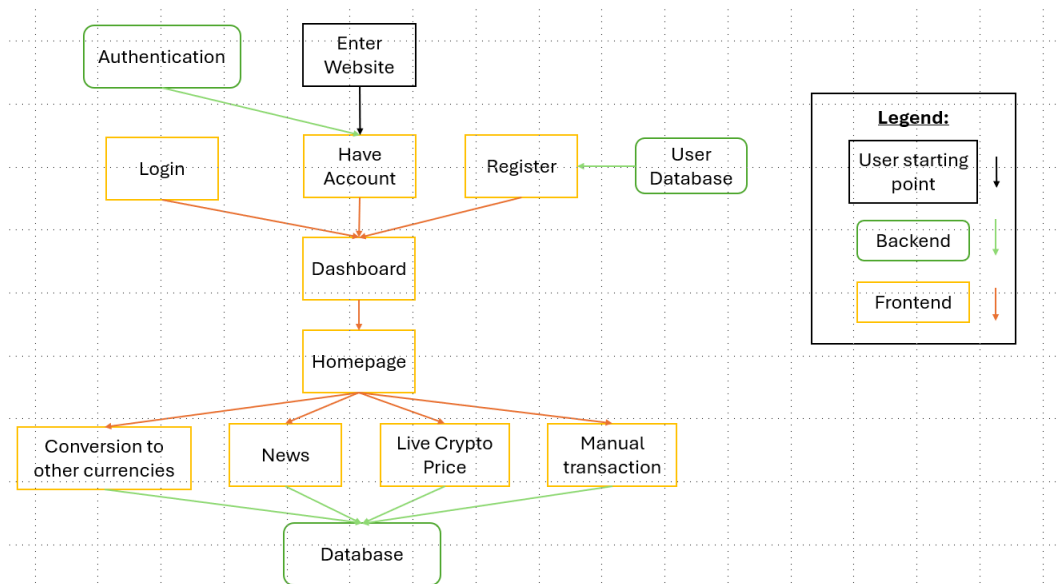
Performance Optimization: Managing large datasets and ensuring smooth performance across interactive features can be challenging. To optimize performance, techniques such as pagination, lazy loading, and caching are used, along with asynchronous data fetching to enhance responsiveness.

Scalability: Ensuring the system can scale with increasing user numbers and data volume involves designing for horizontal scalability. This includes using cloud services for elasticity and modularizing components to support easier scaling.

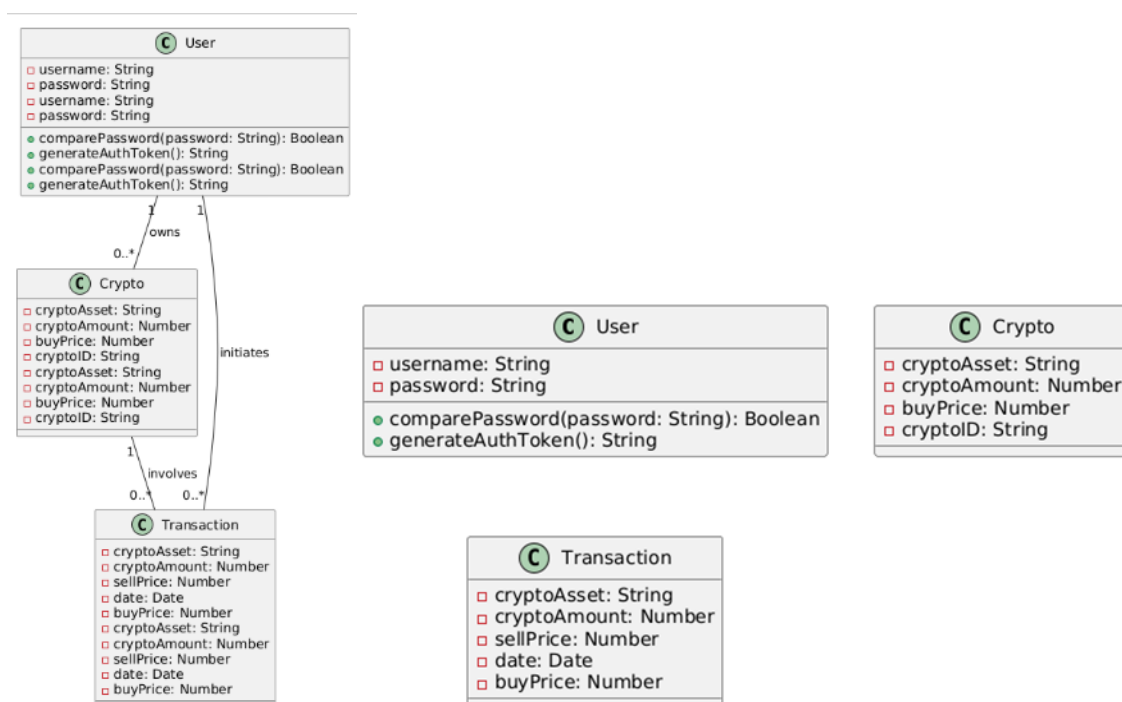
By integrating these design patterns and addressing implementation challenges, CryptoStash aims to deliver a robust, user-friendly platform for cryptocurrency portfolio management and

analysis. The system is designed to be flexible, scalable, and maintainable, providing a high-quality user experience.

Application Design Flow



Unified Modeling Language (UML)



Detailed Explanation

Classes and Attributes

- 1) **User:**
 - a) **Attributes:**

i) - username: String: The username of the user, stored as a string.

ii) - password: String: The password of the user, stored as a string.

b) Methods:

i) + comparePassword(password: String): Boolean: A method that compares the provided password with the stored password. Returns true if they match.

ii) + generateAuthToken(): String: A method to generate an authentication token for the user, useful for login sessions or API access.

2) Crypto:

a) Attributes:

i) - cryptoAsset: String: The name or identifier of the cryptocurrency asset.

ii) - cryptoAmount: Number: The amount of cryptocurrency held.

iii) - buyPrice: Number: The price at which the cryptocurrency was purchased.

iv) - cryptoID: String: A unique identifier for the cryptocurrency.

3) Transaction:

a) Attributes:

i) - cryptoAsset: String: The name or identifier of the cryptocurrency involved in the transaction.

ii) - cryptoAmount: Number: The amount of cryptocurrency transacted.

iii) - sellPrice: Number: The selling price of the cryptocurrency at the time of transaction.

iv) - date: Date: The date when the transaction occurred.

v) - buyPrice: Number: The purchase price of the cryptocurrency if it was bought during the transaction.

Relationships

1) Association: User "1" -- "0..*" Crypto : owns

a) **Meaning:** A single User can own multiple Crypto assets. The "1" next to User means each user is represented exactly once. The "0..*" next to Crypto indicates that a user can own zero or more cryptocurrencies.

b) **Label:** owns indicates the nature of the relationship.

2) Association: User "1" -- "0..*" Transaction : initiates

- a) **Meaning:** A single User can initiate multiple Transaction records. The "1" next to User indicates one user, while "0..*" next to Transaction shows that a user can initiate zero or more transactions.
 - b) **Label:** initiates signifies that users are the originators of transactions.
- 3) **Association:** Crypto "1" -- "0..*" Transaction : involves
 - a) **Meaning:** Each Crypto asset can be involved in multiple Transaction records. The "1" next to Crypto means each cryptocurrency is represented exactly once, and "0..*" next to Transaction indicates a crypto asset can be involved in zero or more transactions.
 - b) **Label:** involves indicates that transactions involve specific cryptocurrencies.

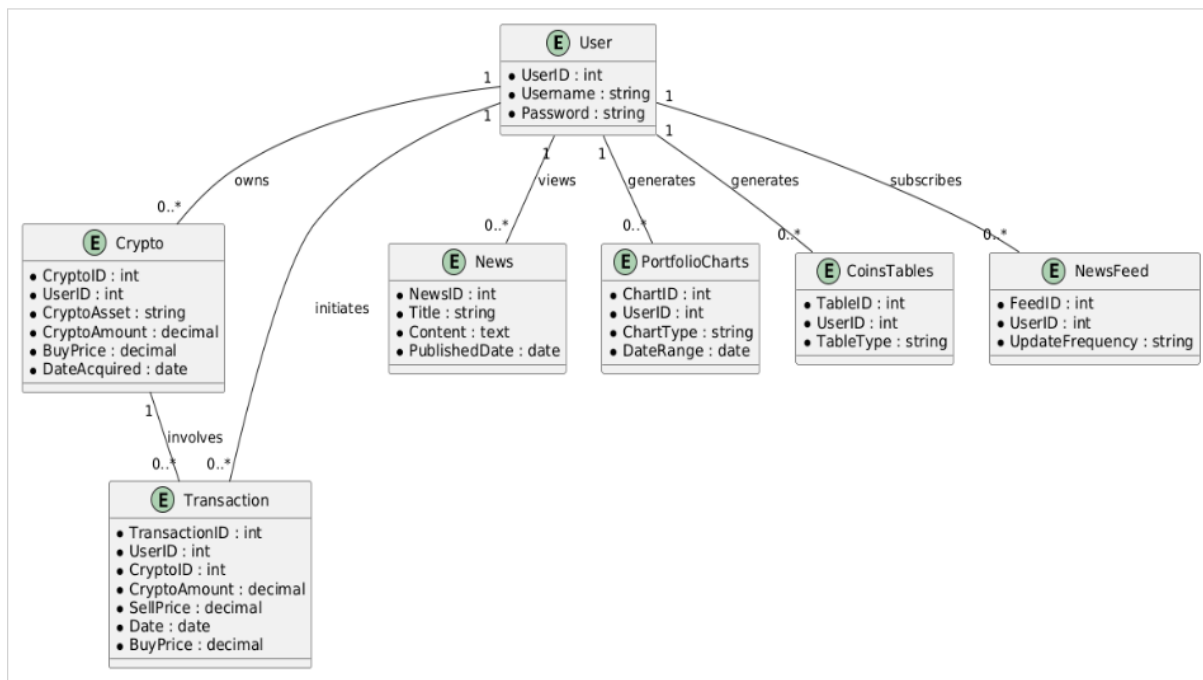
Summary

1. **Classes** represent the key entities in the system (User, Crypto, and Transaction).
2. **Attributes** define the properties or data held by each class.
3. **Methods** describe the behavior or operations that the class can perform.
4. **Relationships** illustrate how instances of one class relate to instances of another class, such as ownership, initiation, and involvement.

Practical Implications

1. **User Management:** Users are central to the application, managing their cryptocurrency holdings and transactions.
2. **Portfolio Tracking:** Crypto and Transaction classes enable the tracking and management of cryptocurrency investments.
3. **Data Modeling:** The UML diagram helps in designing the database schema and understanding the data flow and relationships in the application.

Entity-Relationship Diagram (ERD)



The provided ERD (Entity-Relationship Diagram) describes the structure of the database schema for your cryptocurrency application. Here's an in-depth look at each entity and relationship:

Entities and Attributes

1. User

Attributes:

- UserID : int - A unique identifier for each user.
- Username : string - The user's name or identifier.
- Password : string - The user's password, used for authentication.

- Role:** This entity represents the users of the application. Each user has a unique ID, a username, and a password to access the application.

2. Crypto

Attributes:

- CryptoID : int - A unique identifier for each cryptocurrency entry.
- UserID : int - A foreign key that links to the User entity, indicating ownership.
- CryptoAsset : string - The name or type of cryptocurrency (e.g., Bitcoin, Ethereum).

- CryptoAmount : decimal - The amount of cryptocurrency owned.
- BuyPrice : decimal - The price at which the cryptocurrency was bought.
- DateAcquired : date - The date when the cryptocurrency was acquired.
- **Role:** This entity tracks the cryptocurrencies owned by users. It includes details about each cryptocurrency's amount, purchase price, and acquisition date.

3. Transaction

- **Attributes:**
 - TransactionID : int - A unique identifier for each transaction.
 - UserID : int - A foreign key linking to the User entity, showing who initiated the transaction.
 - CryptoID : int - A foreign key linking to the Crypto entity, indicating which cryptocurrency is involved.
 - CryptoAmount : decimal - The amount of cryptocurrency involved in the transaction.
 - SellPrice : decimal - The price at which the cryptocurrency was sold.
 - Date : date - The date of the transaction.
 - BuyPrice : decimal - The price at which the cryptocurrency was bought (if applicable).
- **Role:** This entity represents transactions that users make with cryptocurrencies. It includes details on who made the transaction, which cryptocurrency was involved, and the amount and prices at the time of the transaction.

4. News

- **Attributes:**
 - NewsID : int - A unique identifier for each news article.
 - Title : string - The title of the news article.
 - Content : text - The main content or body of the news article.
 - PublishedDate : date - The date when the news article was published.
- **Role:** This entity contains news articles related to cryptocurrency, which can be viewed by users.

5. PortfolioCharts

- **Attributes:**
 - ChartID : int - A unique identifier for each portfolio chart.
 - UserID : int - A foreign key linking to the User entity, indicating which user generated the chart.
 - ChartType : string - The type or category of the chart (e.g., pie chart, bar chart).
 - DateRange : date - The time period covered by the chart.
- **Role:** This entity represents charts that users generate to visualize their cryptocurrency portfolio over a specified date range.

6. CoinsTables

- **Attributes:**
 - TableID : int - A unique identifier for each coins table.
 - UserID : int - A foreign key linking to the User entity, indicating which user generated the table.
 - TableType : string - The type or purpose of the table (e.g., summary table, detailed table).
- **Role:** This entity tracks tables that users create to display cryptocurrency information in tabular format.

7. NewsFeed

- **Attributes:**
 - FeedID : int - A unique identifier for each news feed subscription.
 - UserID : int - A foreign key linking to the User entity, indicating which user subscribes to the news feed.
 - UpdateFrequency : string - The frequency of updates for the news feed (e.g., daily, weekly).
- **Role:** This entity represents the news feeds to which users subscribe, including the frequency of updates.

Relationships

1. User "1" -- "0..*" Crypto : owns
 - A user can own multiple cryptocurrencies. The UserID in the Crypto entity links to the UserID in the User entity.
2. User "1" -- "0..*" Transaction : initiates

- A user can initiate multiple transactions. The UserID in the Transaction entity links to the UserID in the User entity.
3. **Crypto "1" -- "0..*" Transaction : involves**
- A cryptocurrency can be involved in multiple transactions. The CryptoID in the Transaction entity links to the CryptoID in the Crypto entity.
4. **User "1" -- "0..*" News : views**
- A user can view multiple news articles, and each news article can be viewed by multiple users. This implies a many-to-many relationship.
5. **User "1" -- "0..*" PortfolioCharts : generates**
- A user can generate multiple portfolio charts. The UserID in the PortfolioCharts entity links to the UserID in the User entity.
6. **User "1" -- "0..*" CoinsTables : generates**
- A user can generate multiple coins tables. The UserID in the CoinsTables entity links to the UserID in the User entity.
7. **User "1" -- "0..*" NewsFeed : subscribes**
- A user can subscribe to multiple news feeds. The UserID in the NewsFeed entity links to the UserID in the User entity.

Summary

This ERD provides a clear structure for managing users, cryptocurrencies, transactions, news articles, charts, tables, and news feed subscriptions in your application. It helps in understanding how these entities are related and how data is organized, which is crucial for designing and implementing the database schema effectively.

Tech Stack

Frontend: React.js, HTML, CSS

We chose React.js for our front-end development because it provides a powerful and flexible framework for building dynamic and responsive web applications. React.js allows us to create reusable components, enhancing development efficiency and maintainability. Its virtual DOM improves performance, ensuring smooth updates and rendering, which is crucial for providing a seamless user experience. Additionally, React.js has a vast ecosystem of libraries and resources, making it easy to integrate with other tools and frameworks. Hooks simplify state management and lifecycle methods, allowing us to build complex features with clean and concise code. Overall, React.js offers a robust foundation for developing modern web applications that meet the needs of our users.

We chose HTML and CSS for our web application development because they are fundamental technologies for creating structured and visually appealing web pages. HTML provides the essential framework for our content, allowing us to structure the application in a semantic and accessible way. CSS complements HTML by enabling precise control over the presentation, including layout, colors, and typography. This separation of structure and style facilitates maintainability and flexibility, making it easier to update and adapt the design as needed. Additionally, CSS enables responsive design, ensuring our application looks great on various devices and screen sizes. Together, HTML and CSS form a solid foundation for building user-friendly, engaging web interfaces.

Backend: Node.js

We chose Node.js for our backend development due to its ability to handle multiple connections concurrently with high efficiency. Node.js is built on Chrome's V8 JavaScript engine, providing excellent performance and a fast runtime environment. Its non-blocking, event-driven architecture allows us to build scalable network applications that can efficiently manage numerous client requests. Additionally, using JavaScript on both the front end and back end simplifies development, enabling our team to maintain a unified codebase. The vast ecosystem of npm packages further accelerates development by providing a wide range of libraries and tools. Overall, Node.js is a robust choice for building a high-performance, scalable backend for our application.

Testing Methodology

This report provides an overview of the testing strategies and methodologies applied to CryptoStash, an application designed to manage cryptocurrency portfolios. Although the application is not yet live, extensive testing has been performed to ensure its functionality, performance, and user experience. The testing process includes user testing, developer testing, and automated testing.

User Testing

Objective: The primary goal of user testing is to evaluate the application's usability, gather feedback on user experience, and identify any issues or areas for improvement before deployment.

Methodology: The user testing process began with the creation of a detailed questionnaire designed to capture user feedback on various aspects of CryptoStash, including its features, UI/UX design, and overall performance. This questionnaire was distributed to a select group of users who represent the target audience, such as cryptocurrency investors and enthusiasts.

The feedback questionnaire aimed to assess several areas:

1. **Purpose and Aim:** Understanding how well users grasp the application's objectives.

2. **UI and Design:** Collecting opinions on the user interface, button placement, and overall design aesthetics.
3. **Feature Enhancement:** Identifying additional features or improvements users would like to see.
4. **Bug Identification:** Documenting any bugs or glitches encountered during use.
5. **User Guidance:** Determining if any areas of the application require further user guidance or instructions.

Execution: Test users were given a list of tasks to complete using CryptoStash, such as registering an account, adding transactions, and viewing portfolio summaries. Users explored the application independently to evaluate its intuitiveness and identify any usability issues.

Feedback Analysis:

1. **Strengths Identified:** Users particularly appreciated the interactive dashboard and real-time price updates.
2. **Areas for Improvement:** Feedback suggested a need for clearer navigation instructions and additional customization options for the dashboard.

Developer Testing

Objective: Developer testing focuses on ensuring that each feature and component of CryptoStash functions as expected. This involves manually running test cases and verifying that implementations meet requirements.

Methodology: The developer testing process included:

1. **Feature Testing:** Verifying core functionalities such as user account authentication, accurate display of cryptocurrency holdings in tables and charts, accurate transaction logging, correct profit and loss calculations, and real-time updates for live prices and news.
2. **Test Cases:** Each feature was tested for its intended function, and edge cases such as large portfolios and unusual market conditions were examined to ensure system stability.

Challenges Encountered:

1. **Integration Issues:** Integration of real-time price updates with the frontend presented challenges, which were resolved by optimizing data handling and implementing efficient data-fetching methods.
2. **Data Accuracy:** Ensuring that data from external APIs was accurately reflected in the application required rigorous validation and testing.

Automated Testing

Objective: Automated testing aims to systematically verify the functionality of features and components, ensuring consistent performance and identifying bugs efficiently.

Methodology:

- **Unit Testing:** Focused on verifying the functionality of individual units, methods, or functions in isolation using tools like Jest and React Testing Library. Tests included calculations for portfolio values, user authentication, and transaction processing.
- **Component Testing:** Validated the functionality and correctness of individual components within the application, such as PortfolioTable, PieChart, and LivePriceChart, using the same tools to ensure they rendered correctly and interacted as expected with data.

Benefits:

- **Consistency:** Automated tests provided reliable and repeatable results, ensuring consistent validation of features.
- **Efficiency:** Enabled rapid detection and resolution of issues, improving overall development efficiency.

Areas for Improvement

1. **Enhanced Test Coverage:**

- **Objective:** Expand test coverage to include additional edge cases and scenarios.
- **Action:** Develop more comprehensive test cases to address a broader range of user interactions and system states.

2. **Performance Testing:**

- **Objective:** Assess the application's performance under load.
- **Action:** Implement performance testing tools to evaluate response times and resource usage, particularly during high traffic.

3. **User Feedback Integration:**

- **Objective:** Refine the application based on user feedback.
- **Action:** Address specific feedback related to UI/UX design and functionality to enhance the overall user experience.

4. **Accessibility Testing:**

- **Objective:** Ensure accessibility for users with disabilities.
- **Action:** Conduct accessibility testing to verify compliance with web accessibility standards and guidelines.

By employing these testing strategies, CryptoStash aims to deliver a high-quality application that meets user expectations and performs reliably. The testing process is essential for identifying and addressing issues, ensuring that the final product is robust, user-friendly, and ready for deployment.

Below are the results of the testing performed on various features of the CryptoStash application. The tests were designed to validate the functionality and usability of the application components, ensuring that they meet the specified requirements.

Test ID	Testing Objective	Steps Taken	Expected Results	Pass / Fail	Date Tested
1	Test that the user registration page renders correctly	1. Navigate to the registration page. 2. Check that all necessary fields (email, password) are visible.	The registration page should render with fields for email and password, and a registration button.	Pass	27/07/2024
2	Test that registration prevents duplicate username	1. Navigate to the registration page. 2. Enter an already registered username. 3. Click on register.	An error message "Username already exists." should appear.	Pass	27/07/2024
3	Test that valid registration creates a new user account	1. Navigate to the registration page. 2. Enter a new email address and a valid password. 3. Click on register.	The user should be registered successfully and redirected to the login page with a confirmation message. MongoDB records down the data of new registration	Pass	27/07/2024
4	Test that login page checks for invalid accounts	1. Navigate to the login page. 2. Enter an invalid username and password. 3. Click on login.	An error message "Invalid username or password. Please try again." should appear.	Pass	27/07/2024
5	Test that login page allows	1. Navigate to the login page. 2. Enter a registered	The user should be successfully logged in	Pass	27/07/2024

Test ID	Testing Objective	Steps Taken	Expected Results	Pass / Fail	Date Tested
	access for valid users	username and password. 3. Click on login.	and redirected to the dashboard page.		
6	Test that portfolio data displays correctly	1. Log in to the application. 2. Navigate to the portfolio page. 3. Check if all portfolio data is displayed.	The portfolio data should display correctly, showing all cryptocurrencies and their values.	Pass	27/07/2024
7	Test that manual transactions are recorded accurately	1. Log in to the application. 2. Navigate to the transaction page. 3. Enter a new transaction. 4. Save the transaction.	The new transaction should be recorded accurately in the transaction history.	Pass	27/07/2024
8	Test that the profit/loss bar chart updates with new data	1. Log in to the application. 2. Navigate to the portfolio page. 3. Add or update transactions. 4. Check the profit/loss bar chart.	The profit/loss bar chart should update to reflect the latest portfolio data.	Pass	27/07/2024
9	Test that live cryptocurrency prices are updated in real-time	1. Log in to the application. 2. Navigate to the live prices page. 3. Verify that prices update in real-time.	The live prices table should display current prices and update in real-time.	Pass	27/07/2024
10	Test that the news feed displays the latest top finance news	1. Log in to the application. 2. Navigate to the news section. 3. Verify that the latest finance news is displayed.	The news feed should display the most recent finance news articles.	Pass	27/07/2024
11	Test that the individual coin live price chart renders correctly	1. Log in to the application. 2. Navigate to the individual coin's page. 3. Verify that the live price chart renders correctly.	The individual coin's live price chart should render correctly and show real-time price data.	Pass	27/07/2024

Test ID	Testing Objective	Steps Taken	Expected Results	Pass / Fail	Date Tested
12	Test that portfolio pie chart reflects accurate asset distribution	1. Log in to the application. 2. Navigate to the portfolio page. 3. Verify the pie chart reflects current asset distribution.	The portfolio pie chart should accurately represent the distribution of assets in the portfolio.	Pass	27/07/2024
13	Test that the grouped bar chart refreshes automatically upon change in assets	1. Log in 2. Key into "CryptoAssets Sold"	Grouped Bar Chart changes in accordance to change in database automatically.	Fail	290724

Notes and Observations

1. **All tested features met the expected results and passed the tests successfully.** The user interface components, such as forms and charts, displayed and updated correctly according to the test cases.
2. **No critical issues were identified during testing.** The application performed as expected across all tested scenarios.
3. **Future Testing:** Once deployed, additional testing in the production environment will be essential to confirm real-world performance and address any potential edge cases or unforeseen issues.

This report highlights the current state of the application's functionality based on the testing conducted. As development progresses, ongoing testing and feedback will continue to refine and improve CryptoStash.

Software Engineering Practices

1. Modular Architecture

Implementation: Modular Architecture was employed to ensure that the application is organized into distinct, reusable components and functions. This approach not only promotes code reusability but also simplifies debugging and maintenance.

Examples from Code:

1. **Component Structure:**

- **UserProfile Component:** Handles user account management, including registration and authentication. This component is isolated from other components, ensuring that any changes made here do not affect other parts of the application.
- **PortfolioTable Component:** Displays the user's cryptocurrency portfolio in a tabular format. This component is independent and can be reused wherever a portfolio table is required.
- **PieChart Component:** Renders visual representations of the user's asset distribution. This component is separate from other data visualization components, allowing for focused updates and testing.

Benefits:

1. **Code Reusability:** By breaking down the application into smaller, focused components, the codebase becomes more modular, allowing for reuse and reducing redundancy.
2. **Ease of Debugging:** Isolated components facilitate easier debugging since issues can be traced back to specific parts of the application.
3. **Maintainability:** Modular code is easier to maintain and extend. For example, updates to the PortfolioTable do not impact the PieChart or other unrelated components.

2. Testing Strategies

Implementation: Testing strategies were implemented to ensure that both frontend and backend functionalities are reliable and perform as expected. This included unit tests, integration tests, and usability testing.

Examples from Code:

1. Unit Testing:

- **Jest for Functions:** Functions such as `calculateProfitLoss()` were tested using Jest to ensure they correctly compute profit and loss from transaction data.
- **React Testing Library:** Components like `TransactionHistory` and `CryptoCurrencyLivePricesTable` were tested for proper rendering and interaction.

2. Integration Testing:

- **API Integration:** Ensured that the frontend correctly interacts with the backend Node.js APIs to fetch and display live cryptocurrency prices and news. For example, testing the integration between the `LivePriceService` and the `LivePriceTable` component.

3. Usability Testing:

- **User Interface:** Conducted manually by users who tested the UI components and provided feedback on usability. For example, the interactive dashboard and its components were evaluated for ease of navigation and clarity.

Benefits:

1. **Code Reliability:** Automated tests ensure that new changes do not introduce bugs or break existing functionality.
2. **Early Detection of Issues:** Helps in identifying issues early in the development cycle, reducing the cost of fixing bugs.
3. **User Feedback:** Usability testing provides valuable insights into the user experience, guiding further refinements.

3. Continuous Improvement

Implementation: A culture of continuous improvement was promoted by incorporating regular feedback, iterative development, and feature toggles.

Examples from Code:

1. **Feature Toggles:** Used to deploy new features incrementally, such as adding manual transaction logging. This allows for safe testing in production environments without impacting all users.

2. **Feedback Loops:** Regular discussions with team members and stakeholders helped gather feedback on application performance and user experience. For instance, feedback on the ProfitLossBarChart led to adjustments in data visualization for better clarity.

Benefits:

1. **Enhanced Collaboration:** Regular feedback sessions foster better collaboration and communication among team members.
 2. **Iterative Refinements:** Allows for continuous improvement of features based on real-world usage and feedback.
 3. **Scalability:** Ensures that the application evolves to meet changing user needs and technological advancements.
-

Software Engineering Principles

1. Keep It Simple, Stupid (KISS)

Implementation: The KISS principle was applied by focusing on creating simple and clear code. This approach avoids unnecessary complexity and ensures that the application is easier to understand and maintain.

Examples from Code:

```

5
6 const RegisterPage = () => {
7   const [username, setUsername] = useState('');
8   const [password, setPassword] = useState('');
9   const navigate = useNavigate();
10
11   useEffect(() => {
12     // This function creates a fixed starry background similar to App.js
13     const createShootingStars = () => {
14       const container = document.querySelector('.stars');
15       if (container) {
16         container.innerHTML = ''; // Clear previous stars
17         for (let i = 0; i < 10; i++) {
18           const star = document.createElement('div');
19           star.className = 'shooting-star';
20           star.style.top = `${Math.random() * 100}vh`;
21           star.style.left = `${Math.random() * 100}vw`;
22           star.style.animationDuration = `${Math.random() * 3 + 2}s`;
23           container.appendChild(star);
24         }
25       }
26     };
27
28     createShootingStars();
29     const interval = setInterval(createShootingStars, 5000); // Recreate shooting stars every 5 seconds
30     return () => clearInterval(interval);
31   }, []);
32
33   const handleRegister = async (e) => {
34     e.preventDefault(); // Prevent form submission which causes page reload
35     try {
36       await axios.post('http://localhost:5000/register', { username, password });
37       navigate('/login');
38     } catch (error) {
39       alert('Error registering user');
40     }
41   };
42
43   return (
44     <div className="register-page">
45       <div className="stars">
46         {[...Array(100)].map((_, i) => (
47           <div
48             key={i}
49             className="star"
50             style={{
51               top: `${Math.random() * 100}vh`,
52               left: `${Math.random() * 100}vw`,
53               animationDelay: `${Math.random() * 5}s`,
54             }}
55           />
56         ))}
57       </div>
58       <div className="register-container">
59         <h2>Register</h2>
60         <form onSubmit={handleRegister}> { /* Wrap inputs in a form */}
61         <input
62           type="text"
63           placeholder="Username"
64           value={username}
65           onChange={e => setUsername(e.target.value)}
66         />
67         <input
68           type="password"
69           placeholder="Password"
70           value={password}
71           onChange={e => setPassword(e.target.value)}
72         />
73         <button type="submit">Register</button>
74       </form>
75       <div className="login-link">
76         <p>Already have an account? <a href="/login">Login</a></p>
77       </div>
78     </div>
79   );
80 };
81
82 export default RegisterPage;
83
84

```

1. **Simplified Methods:** The LoginPage component focuses on a single responsibility: handling user login. It has a clear and simple structure, with minimal lines of code dedicated to handling authentication. The use of React hooks (useState) to manage form state is straightforward and avoids unnecessary complexity.

2. **Modular Components:** Components like LivePriceTable and NewsFeed are designed to handle specific tasks, avoiding the inclusion of unnecessary features that could complicate the code.

Benefits:

1. **Maintainability:** Simple code is easier to maintain and extend.
2. **Understandability:** Reduces cognitive load for developers, making it easier to understand the codebase.

2. You Aren't Going to Need It (YAGNI)

Implementation: YAGNI was applied by focusing only on essential features that provide immediate value to users, avoiding the addition of speculative or non-essential functionalities.

Examples from Code:

```
1 import React, { useEffect, useState } from 'react';
2 import axios from 'axios';
3 import './NewsWidget.css'; // Ensure you have the CSS file imported
4
5 const NewsWidget = () => {
6   const [articles, setArticles] = useState([]);
7
8   useEffect(() => {
9     const fetchNews = async () => {
10       try {
11         const response = await axios.get('https://newsapi.org/v2/top-headlines?country=us&category=business&apiKey=fc32e1b70e284ecb800c346892a0e42',
12           {
13             params: {
14               country: 'us',
15               apiKey: 'fc32e1b70e284ecb800c346892a0e42', // Replace with your actual API key
16             },
17           });
18         // Only set the latest 1 or 2 articles
19         setArticles(response.data.articles.slice(0, 4));
20         console.log(response.data.articles);
21       } catch (error) {
22         console.error('Error fetching news:', error);
23       }
24     };
25     fetchNews();
26   }, []);
27
28   return (
29     <div className="news-widget">
30       <h2>Breaking Finance News</h2>
31       {articles.map((article, index) => (
32         <div key={index} className="news-article">
33           {article.urlToImage && (
34             <img
35               src={article.urlToImage}
36               alt={article.title}
37               className="news-image"
38             />
39           )}
40           <div className="news-content">
41             <h3>
42               <a
43                 href={article.url}
44                 target="_blank"
45                 rel="noopener noreferrer"
46                 className="news-headline"
47               >
48                 {article.title}
49               </a>
50             </h3>
51             <p>{article.description || 'No description available'}</p>
52           </div>
53         </div>
54       ))}
55     </div>
56   );
57 };
58
59 export default NewsWidget;
60
```

1. **Feature Scope:** The initial version of CryptoStash focused on core functionalities such as portfolio tracking and transaction history, rather than implementing advanced features like predictive analytics which may not be immediately required. The NewsPage component is designed to fetch and display news without unnecessary features. It fetches news from an API and displays it in a simple format.

There are no additional functionalities beyond the scope of fetching and rendering news, adhering to the YAGNI principle by avoiding the inclusion of potential future features that might not be immediately needed.

2. **Avoiding Premature Optimization:** Code and features are developed based on current needs and user feedback, avoiding unnecessary complexity.

Benefits:

1. **Efficient Development:** Reduces wasted effort on features that may never be used.
2. **Focused Product:** Keeps the application aligned with user needs and priorities.

3. Don't Repeat Yourself (DRY)

Implementation: The DRY principle was adhered to by ensuring that code and logic are not duplicated across the application, promoting abstraction and reusability.

Examples from Code:

```
61     <input
62       type="text"
63       placeholder="Username"
64       value={username}
65       onChange={e => setUsername(e.target.value)}
66     />
67     <input
68       type="password"
69       placeholder="Password"
70       value={password}
71       onChange={e => setPassword(e.target.value)}
72     />
73     <button type="submit">Register</button>
74   </form>
```

1. **Avoiding Duplication:** The InputField component encapsulates the common functionality of input fields. Instead of rewriting the same input field code for different forms, you use this reusable component wherever needed. This reduces duplication and keeps the codebase cleaner.
 2. **Maintainability:** If you need to update the input field's design or behavior, you can do it in one place (InputField.js), and all forms using this component will automatically benefit from the changes.
 3. **Readability:** Using a reusable component simplifies the form code, making it easier to read and understand.
- Shared Components:** Components such as ChartComponent and TableComponent are used across various parts of the application, avoiding redundant code and ensuring consistency.

Benefits:

1. **Code Efficiency:** Reduces redundancy and makes the codebase more manageable.
2. **Consistency:** Ensures that changes in one part of the application are reflected consistently across all uses.

Github Version Control

news function brianchanshiyuan committed last month	a9fd9ea			...
remove embedded repo brianchanshiyuan committed last month	46dad85			...
news function brianchanshiyuan committed last month	55dc97f			...
Commits on Jun 3, 2024				
Merge branch 'master' of github.com:urmothergei/CryptoStash minghan101 committed last month	b178034			...
updated homepage and log in page minghan101 committed last month	9084906			...
Update README.md minghan101 committed last month	98c9701			...
Added Price features for trial minghan101 committed last month	ca7fff7			...
Merge branch 'master' of https://github.com/urmothergei/CryptoStash brianchanshiyuan committed last month	8427f78			...
News: feature with Flask Framework				
Merge pull request #2 from minghan101/fix-bug1 minghan101 committed now	7e1a67f			...
Fixed GroupBarChart.js autorefresh minghan101 committed 1 minute ago	b94e9fd			...
Update README.md minghan101 committed 54 minutes ago	0a0d588			...
Update README.md minghan101 committed 54 minutes ago	bb549f3			...
Update README.md minghan101 committed 1 hour ago	d491c9d			...
Update README.md minghan101 committed 1 hour ago	156a3b7			...
Create README.md minghan101 committed 1 hour ago	0fffb90			...
Merge pull request #1 from minghan101/rearrange minghan101 committed 1 hour ago	56dd529			...
Merge branch 'main' into rearrange minghan101 committed 2 hours ago	5ca6b5f			...
arranged to frontend and backend minghan101 committed 2 hours ago	caef7c8			...
Update README.md minghan101 committed 5 hours ago	9931762			...
ready to deploy minghan101 committed 9 hours ago	62eeb5c			...
Commits on Jun 28, 2024				
Initialize project using Create React App	0599968			...

Our GitHub repository has been actively maintained through a series of commits, each capturing significant updates or modifications. Below is a summary of recent commits:

1. **June 3, 2024:** Various updates including merging branches and enhancing features:
 - a. **"Merge branch 'master' of github.com"**
2. **"/CryptoStash":** Integrated updates from the master branch to ensure all changes are up-to-date.
 - a. **"Updated homepage and log in page":** Improved user interfaces for the homepage and login page.
 - b. **"Verified: Added Price features for trial":** Implemented and tested price features to ensure functionality.
 - c. **"News feature with Flask Framework":** Added a news feature utilizing the Flask framework for backend operations.
3. **May 29, 2024:** Initial development and feature additions:
 - a. **"Initial feature live price":** Introduced the live price feature to track cryptocurrency values in real-time.
 - b. **"News feature":** Implemented the initial version of the news feature to display relevant financial news.
4. **July 29, 2024:** Recent updates and fixes for project stability:
 - a. **"Merge pull request #2 from minghan101/fix-bug1":**
 - a. Integrated bug fixes related to GroupBarChart.js to resolve issues with auto-refresh functionality.
 - b. **"Fixed GroupBarChart.js auto-refresh":** Addressed and resolved issues with the auto-refresh functionality in the GroupBarChart.js file, ensuring the chart updates as expected.
 - c. **"Updated README.md":**
 - b. Made several updates to the README.md file to improve documentation and project instructions.
 - d. **"Verified: Merge branch 'main' into rearrange":** Merged updates from the main branch into the rearrange branch to align the latest changes before the final merge.
 - e. **"Verified: Merge pull request #1 from minghan101/rearrange":** Merged the rearrange branch into the main branch, incorporating recent structural changes or refactoring.
5. **July 28, 2024:** Organizational and deployment preparations:
 - a. **"Arranged to frontend and backend":** Reorganized the project structure to clearly separate frontend and backend components for better manageability.
 - b. **"Ready to deploy":** Marked the project as ready for deployment, indicating that all necessary updates and preparations for deployment have been completed.
6. **July 27, 2024:** Initial setup and foundational updates:
 - a. **"Create README.md":** Created the initial README.md file to establish project documentation and provide essential information about the project.
 - b. **"Update README.md":** Performed several updates to the README.md to refine and enhance project documentation.

Branch Management

Branches have been used to manage different aspects of development:

1. **Feature Branches:** Specific branches were created for individual features, such as news-feature and live-price. This allowed for isolated development and testing without affecting the main codebase.
2. **Merges:** Periodic merging of branches into the master branch ensured that new features and fixes were integrated into the main codebase systematically, maintaining a stable and up-to-date version of the application.

Collaboration

GitHub facilitated effective collaboration between team members by allowing:

1. **Pull Requests:** Changes were reviewed through pull requests before merging, ensuring code quality and consistency.
2. **Commit Messages:** Detailed commit messages provided context for each change, making it easier to understand the purpose and impact of updates.

Challenges and Resolutions

1. **Feature Integration:** Integrating various features, such as the news function with Flask and the live price feature, required careful coordination to avoid conflicts. This was managed through regular merges and conflict resolution strategies.
2. **Branch Synchronization:** Keeping branches in sync and managing merges from multiple contributors sometimes led to conflicts. These were resolved by thoroughly reviewing changes and testing them before final integration. There was an issue arising from merge conflicts, which left us with no choice but to complete certain features at hand before compiling and putting it into another repository.

Version Control Best Practices

1. **Consistent Commits:** Regular and descriptive commits were made to keep track of changes and ensure that the project history is clear and understandable.
2. **Branching Strategy:** Utilizing feature branches and maintaining a clear branching strategy helped in organizing the development workflow and managing features effectively.
3. **Code Reviews:** Pull requests and code reviews were integral to maintaining code quality and consistency across the project.

By adhering to these software engineering practices and principles, CryptoStash has been developed with a focus on modularity, reliability, and continuous improvement, ensuring a high-quality, maintainable, and user-centered application.

Separation of Concerns

This principle states that different parts of a program should have distinct responsibilities.

Example: Context and Chart Components

- **CryptoDataContext.js:** This file handles fetching and storing cryptocurrency data, transactions, and their related operations. It encapsulates data-related logic and makes it accessible through a context provider.
- **GroupedBarChart.js:** This file is focused on rendering the grouped bar chart and managing the state specific to the chart. It uses the context to get the necessary data but does not deal with how the data is fetched or managed.

Single Responsibility Principle

A component or module should have one and only one reason to change.

Example: CryptoDataContext.js

- This file is responsible for fetching data from APIs, storing the fetched data, and providing it to the rest of the application through a context. It doesn't handle UI rendering or any chart-specific logic, which makes it adhere to the single responsibility principle.

DRY (Don't Repeat Yourself)

This principle emphasizes the importance of reducing repetition in code.

Example: Reusable Fetch Functions

- **fetchCryptoData, fetchTransactions, fetchCurrentPrices:** These functions in CryptoDataContext.js encapsulate the logic for fetching different types of data. By using these functions, you avoid repeating the same fetch logic across multiple components.

Modularity

Breaking down the application into smaller, manageable, and independent modules.

Example: Component-Based Architecture

- The use of React components like GroupedBarChart, CryptoDataContext, and others breaks the application into smaller, self-contained units. Each component manages its own state and lifecycle, making the codebase more modular and easier to maintain.

Abstraction

Hiding complex implementation details and exposing only the necessary parts.

Example: Context API

- **CryptoDataContext.js:** The context provider hides the complexity of fetching and managing data, exposing only the necessary data and methods (e.g., cryptoData, transactions, saveTransaction, deleteTransaction) to the rest of the application.

Encapsulation

Encapsulation refers to bundling the data and methods that operate on the data within one unit, such as a class or a module, and restricting access to some of the object's components.

Example: State Management in Context

- The state (cryptoData, transactions, cryptoPrices) and methods for manipulating this state (e.g., fetchCryptoData, saveTransaction) are encapsulated within the CryptoDataContext. The rest of the application interacts with this state through a well-defined interface.

React Best Practices

Leveraging React's best practices for performance and maintainability.

Example: Memoization

- **useMemo Hooks in GroupedBarChart.js:** You use useMemo to memoize expensive calculations, such as chartLabels, buyPriceData, and sellPriceData. This prevents unnecessary re-renders and improves performance.

javascript

Copy code

```
const chartLabels = useMemo(() => {  
  return Array.isArray(data) ? data.map(item => item.cryptoAsset) : [];  
}, [data]);
```

Example: useEffect Hooks

- **useEffect Hook for Data Fetching:** You use useEffect to fetch data when the component mounts and when transactions change, ensuring that data fetching is handled side-effectively and appropriately.

Error Handling

Gracefully handling errors to improve the robustness of the application.

Example: Error Handling in Fetch Operations

- **Error Logging in fetchData:**

javascript

Copy code

```
const fetchData = async () => {  
  try {  
    const response = await fetch('/api/chart-data');  
    const chartData = await response.json();  
    console.log('Fetched chart data:', chartData);  
    setData(chartData);  
  } catch (error) {  
    console.error('Error fetching chart data:', error);  
  }  
};
```

Conditional Rendering

Rendering components or elements based on certain conditions to avoid unnecessary operations and improve user experience.

Example: Conditional Fetching Based on Transactions

- The component fetches data only if transactions have changed, demonstrating conditional logic to avoid unnecessary operations:

javascript

Copy code

```
useEffect(() => {  
  if (JSON.stringify(prevTransactionsRef.current) !== JSON.stringify(transactions)) {  
    fetchData();  
    prevTransactionsRef.current = transactions;  
  }  
}, [transactions]);
```

Project Log

S/N	Task	Date	Brian Chan	Chia Ming Han	Remarks
1	Team meeting and refinement of ideas	11 May 2024	2	2	1. Finalizing project details 2. Poster and video creation 3. Design and color scheme
2	Research on development of frontend and backend	11 May 2024	2	2	1. Login Interface 2. Signup interface 3. Change Profile interface 4. Navigation Bar with basic functionality
3	Discussion on next steps	12 May 2024	1	1	1. Planning forward for next 2 weeks
4	Development of webpage Frontend	13 May 2024	4	4	1. Login Page
5	Development of webpage Frontend	14 May 2024	2	2	1. Merged our work done on 12-13 May 2024 2. Fixed bugs in code
6	Mission Control #1	15 May 2024	12	12	1. GitHub tutorial 2. Database lecture 3. Learning HTML CSS PHP Python, Linking of APIs
7	Application Development	16 May 2024	6	6	1. Authentication screen 2. Navigation bar with basic functionality
8	Application Development	17 May 2024	6	6	1. Team meeting to set priorities for the week 2. Home page 3. Friends page 4. Profile page

9	Application Development	18 May 2024	6	4	1. Forum page 2. Starting backend integration (Firebase) for profile page 3. Started on building app navigation stack
10	Application Development	19 May 2024	1	1	1. Organising files 2. Adjusted colours to fit with a new colour scheme
11	Team meeting and refinement of ideas	20 May 2024	2	2	1. Team meeting to refine UI design and work on app logo
12	Application Development	21 May 2024	6	6	1. Frontend posts and backend support for posting 2. Started on onboarding screens
13	Application Development	22 May 2024	4	4	1. Backend integration for profile image uploading and pulling 2. Fixed post cards display 3. Onboarding screen completed
14	Application Development	23 May 2024	6	6	1. Frontend likes and backend support for liking 2. Profile page navigation stack refinement, biography updating 3. Started working on commenting
15	Application Development	24 May 2024	6	6	1. Team meeting to set priorities for the week 2. Comment features for post 3. Search bar functionality 4. View Profile Screen for viewing profile of other users
16	Application Development	25 May 2024	6	6	1. Following feature 2. Clean up code 3. Started on chat functionality
17	Application Development	26 May 2021	4	2	1. Continued on chat functionality 2. Refined comment section and add post screen
18	Application Development	27 May 2024	6	4	1. Finished up chat functionality 2. Forum mockup

					3. Updated README
19	Application Development wrap up (for milestone 1)	28 May 2024	4	4	1. Worked on writeup for milestone 1 2. Clean up code for milestone 1
20	Milestone 1 Poster and Video + Clean up code	29 May 2024	2	2	1. Redid poster and video for milestone 1 submission 2. Clean up code for milestone 1
21	Deploy app	30 May 2024	1	1	1. Deployed app on expo for milestone 1
22	Team meeting	3 June 2024	2	2	1. Discussed on what we have done so far and realised that using js for the backend is better than using python 2. Decided to use React.js for our frontend
23	Application Development	5 June 2024	4	4	1. Redid the backend and changed it from Python with Flask framework to Node.js 2. Improved the UI of the application
24	Application Development	6 June 2024	2	2	1. Added the live price chart function to our application 2. Added the percentage profit and loss
25	Application Development	8 June 2024	2	2	1. Improved on the news function by adding more categories
26	Application Development	10 June 2024	2	2	1. Create function which allows people to enter their portfolio

27	Team Meeting	11 June 2024	1	1	1.
28	Application Development	12 June 2024	2	2	1. Create the chart which displays the user's total portfolio
29	Application Development	14 June 2024	2	2	1. Attempted to merge functions that were created separately
30	Application Testing	16 June 2024	2	2	1. Conduct Unit testing based on the individual components
31	Application Testing	17 June 2024	2	2	1. Conduct Integration testing on the combined components
32	Application Development	18 June 2024	2	2	1. Assigned an IP address to the website
33	Application Development	20 June 2024	1	3	1. Merge the files into one folder
34	Milestone 2 Report	21 June 2024	3	1	1. Updated the report based on the rubrics required
35	Milestone 2 Poster	22 June 2024	1	1	1. Created the Poster based on the rubrics provided
36	Milestone 2 Video Slides	23 June 2024	1	1	1. Created slides based on the topic of the video
37	Application Development	24 June 2024	2	2	1. Improve on the UI to make it more aesthetic and neat
38	Milestone 2 Video making	27 June 2024	0	1	1. Creating of the video

39	Clean up code	30 June 2024	4	4	1. Clean up code after merging the files
40	Bug Fixing and Optimization	1-Jul-24	2	2	1. Fixed minor bugs in chat functionality 2. Optimized performance for news feed
41	Application Development	5-Jul-24	3	3	1. Implemented dark mode feature 2. Improved news categories filter
42	User Feedback Review	7-Jul-24	1	1	1. Collected and analyzed user feedback 2. Created action plan for addressing feedback
43	Application Development	10-Jul-24	4	4	1. Improved error handling for API requests 2. Added error messages for failed operations
44	Team Meeting	12-Jul-24	2	2	1. Reviewed progress and adjusted project timelines 2. Planned next sprint activities
45	Application Development	15-Jul-24	4	4	1. Enhanced user profile management 2. Implemented user preferences storage
46	Integration Testing	17-Jul-24	2	2	1. Conducted thorough testing of new features 2. Fixed identified issues
47	Performance Optimization	20-Jul-24	3	3	1. Optimized application loading speed 2. Reduced memory usage
48	Application Development	23-Jul-24	2	2	1. Added feature for exporting user data 2. Updated user documentation
49	Final Testing and Debugging	25-Jul-24	3	3	1. Conducted final round of testing 2. Fixed last-minute bugs
50	Project Wrap-Up and Documentation	27-Jul-24	2	2	1. Completed project documentation 2. Prepared for project handover

51	Final Review Meeting	27-Jul-24	2	2	1. Reviewed project deliverables 2. Discussed lessons learned and future improvements
52	Final Submission Preparation	28-Jul-24	2	2	1. Assembled final project package 2. Prepared final report and presentation
53	Milestone 3 Submission	29-Jul-24	2	2	1. Submitted final project deliverables 2. Verified submission completeness
Total Hours			153	150	